

SAPIENZA
UNIVERSITÀ DI ROMA

Modellazione MIP

Modelling

The methodological part of the course: what a MIP model is, logic and binary variables, the fourteen links between variables with their proofs, relaxations, duality and bounds, constructive heuristics, and the step from the model to Python/Gurobi. The problems on which to apply all this are in notes on the numerical problems and on the problems with a symbolic model.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloads:
fabiofurini.github.io/modellazione-mip/
github.com/fabiofurini/modellazione-mip

Academic year 2026–2027

Contents

1 What is a MIP model **3**

1.1 Data, variables, objective, constraints . . . 3
1.2 Why integrality matters . . . 6
1.3 The LP relaxation and the sandwich of bounds . . . 7
1.4 Three patterns that keep coming back . . . 9
1.5 What the solver does with the two bounds . . . 9
1.6 What this chapter has put in your hands . . . 10

2 Relaxations, duality and bounds **11**

2.1 What a relaxation is . . . 11
2.2 The primal/dual conversion table . . . 12
2.3 Weak duality, strong duality . . . 12
2.4 Three recipes for building a dual solution by hand . . . 13
2.5 A minimisation problem, in full . . . 13
2.6 A maximisation problem: the roles swap . . . 16
2.7 Valid inequalities and constraints that preserve optimality . . . 19
2.8 Stronger formulations: what it means, and how it is proved . . . 20
2.9 What the solver says . . . 20
2.10 LP duals are not the marginal prices of the MILP . . . 21
2.11 The bound protocol . . . 21

3 From the model to Python/Gurobi **23**

3.1 The basic gurobipy instructions . . . 23
3.2 The four classes of variables . . . 25
3.3 The model, one family per block . . . 25
3.4 Reading the results . . . 26
3.5 Tolerances . . . 27
3.6 The relaxation with `relax()` . . . 27
3.7 Three problems the course reuses . . . 28
3.8 The course protocol, from start to finish . . . 33
3.9 Running the code . . . 34

4 Constructive heuristics **36**
4.1 Bin packing: the insertion rules . . . 37
4.2 $P||C_{\max}$: the least loaded rule . . . 38
4.3 Set covering: the cheapest completion rule . . . 39
4.4 Knapsack: the best ratio rule . . . 40
4.5 TSP: the nearest neighbour . . . 40
4.6 Lot sizing: least unit cost period covering . . . 41
4.7 Local search, and what it does not give . . . 42
4.8 When the constructive heuristic fails . . . 43
4.9 What it gives, and what it does not . . . 43
4.10 The overview of the heuristics . . . 44

5 Logic and binary variables **46**
5.1 Propositions and binary variables . . . 46
5.2 Literals, clauses, conjunctive normal form . . . 47
5.3 From CNF to linear constraints . . . 48
5.4 Logical implications . . . 49
5.5 Splitting an implication: when it is allowed . . . 51
5.6 Counting: at most one, at least one, exactly one . . . 51
5.7 Logical operators and set operators . . . 52
5.8 Checking the translation, not trusting it . . . 52
5.9 Five exercises . . . 52
5.10 Logical constraints inside an optimisation model . . . 54
5.11 What this chapter has put in your hands . . . 57

6 Links between variables **58**
6.1 Activation: aggregated and disaggregated form . . . 59
6.2 Fixed cost, capacity and continuous flow . . . 60
6.3 Minimum lot size and semicontinuous variables . . . 61
6.4 Integer counts, multiple capacity, rounding up . . . 62
6.5 The maximum auxiliary variable . . . 62
6.6 Min-max, max-min and the range . . . 63
6.7 The absolute value . . . 64
6.8 Big-M: conditional constraints and disjunctions . . . 65
6.9 Precedences and sequencing . . . 66
6.10 "If and only if" . . . 67
6.11 Counting the different types . . . 68
6.12 Alldiff and binary expansion . . . 68
6.13 Soft constraints, deviations and penalties . . . 69
6.14 Piecewise linear functions . . . 70
6.15 The map of techniques . . . 72

Index **74**

Chapter 1

What is a MIP model

Script: `python/cap01_models.py` Online page

1.1 Data, variables, objective, constraints

A *mathematical programming* model translates a decision into four ingredients, always in the same order:

1. the **data**: the numbers describing the problem, known before deciding (costs, capacities, demands, times…);
2. the **variables**: the quantities the decision maker controls;
3. the **objective**: a function of the variables to minimise or maximise;
4. the **constraints**: the equations and inequalities the variables must satisfy for the solution to be feasible.

An *optimisation* problem asks for the feasible solution (satisfying all constraints) with the best objective value.

The classes of models

- **The acronyms**: upright LP is the *class* of problems, italic LP is *one* problem of that class — the one whose $\tilde{x}(LP)$ and $z(LP)$ are written. The same for ILP, BIP and MILP.
- **The size and the data**: n variables, with $j \in \{1, 2, \ldots, n\}$, and m constraints, with $i \in \{1, 2, \ldots, m\}$; the costs c_j, the coefficients a_{ij} and the right-hand sides b_i.

What a model can contain

- a **linear objective function** of the variables, to be minimised or maximised;
- **linear constraints** of three types: $\geq$, for a requirement to be met; $\leq$, for a capacity to be respected; equality, for a balance or an exact assignment;
- **variables**, in one or more families, each with its own domain, declared at the end of the model.

None of the three has a fixed shape: how many constraints, of which type and with how many families of variables is decided by the problem.

What tells the four classes apart: the domains

- **LP**: all the variables are continuous, $x_j \in \mathbb{R}$, usually non-negative. In this course a model of LP type is never written from scratch: it arrives as the *relaxation* of a MIP model, when the integrality constraints are dropped.
- **ILP**: all the variables are integer, $x_j \in \mathbb{Z}$.
- **BIP**: all the variables are binary, $x_j \in \{0, 1\}$.
- **MILP**: some variables are integer or binary and others continuous.

That is where the difference in difficulty lives: an LP is solved in polynomial time, an ILP and a MILP in general are not.

A model of ILP type

How many units to buy from two suppliers, x_1 and x_2, in whole lots.

$$\begin{aligned} \min \quad & 4x_1 + 7x_2 \\ \text{subject to} \quad & 2x_1 + 3x_2 \geq 12 \\ & x_1 + x_2 \leq 5 \\ & x_1, \quad x_2 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

The objective minimises the spending. The first constraint, of direction $\geq$, requires the demand of 12 units to be covered; the second, of direction $\leq$, caps the total lots at five. The last row is the domain: all the variables are integer, and the model is an ILP.

A model of BIP type

Which projects to fund: $y_k = 1$ if project k is funded, 0 otherwise.

$$\begin{aligned} \max \quad & 5y_1 + 4y_2 + 6y_3 \\ \text{subject to} \quad & y_1 + y_2 + y_3 \leq 2 \\ & y_1 + y_3 \geq 1 \\ & y_1, \quad y_2, \quad y_3 \in \{0, 1\} \end{aligned}$$

The objective maximises the value of the projects chosen. The first constraint, of direction $\leq$, says that at most two of them can be funded; the second, of direction $\geq$, that at least one of the first and the third must be. The last row is the domain: all the variables are binary, and the model is a BIP.

A model of MILP type

Two products to make, with x_1 and x_2 the quantities produced, and a plant to switch on or not, with $y = 1$ if it is switched on and $y = 0$ otherwise.

$$\begin{array}{rrcrcrcl} \max & 3x_1 & + & 8x_2 & - & 10y & & \\ \text{subject to} & x_1 & + & x_2 & & & = & 6 \\ & & & x_2 & - & 4y & \leq & 0 \\ & x_1 & & & & & \geq & 1 \\ & x_1, & & x_2 & & & \geq & 0 \\ & & & & & y & \in & \{0,1\} \end{array}$$

The objective maximises the revenue of the two products minus the cost of the plant. The first constraint, an equality, requires exactly the six units demanded to be produced; the second, of direction $\leq$, says that at most four units of the second product can be made, and only with the plant on, because with $y = 0$ it leaves $x_2 \leq 0$; the third, of direction $\geq$, asks for at least one unit of the first product. The last two rows are the domains: two continuous variables and one binary, and the model is a MILP.

Every datum, every set and every symbol is *defined before it is used*. In every model of these notes:

1. the variables are introduced explicitly before the formulation;
2. the objective and each family of constraints carry an equation number;
3. the domain constraints, which declare the variables, close the model;
4. after the model a list explains objective and constraints family by family.

Notation for optimal values

Optimal values name the model they optimise: $z(MILP)$ is the optimum of a model of MILP type. The relaxations and their duals, introduced later in this chapter, follow the same rule. *Feasible* solutions carry a bar ($\bar{x}$, $\bar{\pi}$), however they were built; *optimal* ones carry a tilde ($\tilde{x}$, $\tilde{\pi}$). The bounds are called LB (lower bound, $LB \leq z(MILP)$) and UB (upper bound, $z(MILP) \leq UB$), whatever the direction of the objective. When the solution a bound comes from matters, it is written in parentheses: a feasible solution $(\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$ of a minimisation model gives

$$UB(\bar{x}) = \sum_{j=1}^{n} c_j \, \bar{x}_j,$$

and in a maximisation model the same sum is $LB(\bar{x})$.

This course works almost exclusively with MILPs: binary or integer variables for "yes/no" or counting decisions, continuous variables for quantities that are measured (time, money, flow).

1.2 Why integrality matters

An ILP or a MILP is not solved by "rounding" the solution of its continuous relaxation. A minimal counterexample shows this.

Rounding fails

$$\begin{array}{lrcrcl} \max & x_1 & + & x_2 & & \\ \text{subject to} & 2x_1 & + & 2x_2 & \leq & 3 \\ & x_1, & & x_2 & \in & \{0,1\} \end{array} \tag{1.1}$$

The LP relaxation replaces $x_1, x_2 \in \{0,1\}$ by $0 \leq x_1, x_2 \leq 1$ and has optimal value $z(LP^+) = 3/2$. Careful: that value is attained by *infinitely many* optimal solutions — every point of the segment $x_1 + x_2 = 3/2$ with $0 \leq x_1, x_2 \leq 1$ — among them (3/4, 3/4), (1, 1/2) and (1/2, 1). Which one the solver returns depends on the algorithm and is not a property of the model: none of them is *the* solution of the relaxation. On our installation Gurobi returns the vertex (1/2, 1).

Rounding gives different outcomes depending on the starting point *and* on the direction. From (3/4, 3/4): upwards it gives (1, 1), which violates the constraint ($2 + 2 = 4 > 3$); downwards it gives (0, 0), feasible but of value 0. From this point *neither* direction finds the optimum. From (1, 1/2): upwards it gives (1, 1) again, infeasible; downwards it gives (1, 0), feasible with value 1, which is exactly the integer optimum $z(MILP) = 1$, attained at $(1, 0)$ and at $(0, 1)$. So there is no "right" direction: on this instance downwards works, and only from one of the two points — and nothing, before solving, told us which of the two the solver would return.

Two distinct lessons. First: rounding can produce *infeasible* points, and when it produces feasible ones there is no guarantee whatsoever on their value. Second: here the difference $3/2 - 1 = 1/2$ is not the fault of rounding — *no* feasible integer point is worth more than 1.

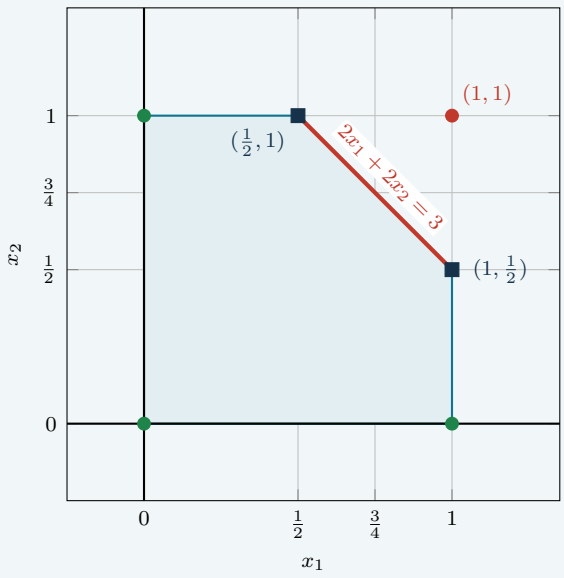


Figure 1.1: Example (1.1). In blue the feasible region of the LP^+ relaxation; in green the three feasible integer points, in red $(1, 1)$, which violates the constraint. The two blue squares are two of the infinitely many optimal solutions of the relaxation, all of value $3/2$.

The lesson: the continuous relaxation gives an indication (a bound, see below), not a solution. MILPs are solved with dedicated algorithms, not with naive heuristics on the continuous problem.

1.3 The LP relaxation and the sandwich of bounds

LP relaxation: two versions not to be confused

The **LP relaxation** of a MILP is obtained by dropping the integrality and binary requirements and keeping everything else. There are two ways of doing this, and the course uses both:

- the **relaxation with the bounds**, $z(LP^+)$: $x \in \{0, 1\}$ becomes $0 \leq x \leq 1$, and $x \in \mathbb{Z}_{\geq 0}$ becomes $x \geq 0$. This is what Gurobi solves with `relax()` and the first relaxation the solver solves;
- the **relaxation without the bounds**, $z(LP)$: $x \in \{0, 1\}$ becomes $x \geq 0$ alone, without carrying over the upper bound. It is weaker than (or equal to) the previous one, but has a more compact dual: this is the one whose dual is written by hand in the exercises.

The relaxation without the bounds is the weaker of the two, that is the bound *furthest* from the integer optimum. The two coincide when the other constraints of the model already imply $x \leq 1$: this happens, for instance, when there is an assignment constraint $\sum_m x_{jm} = 1$. Which side of $z(MILP)$ they sit on is settled by the box below.

The relaxation is a bound: on which side, and why

The relaxation *removes* constraints, so every feasible solution of the MILP stays feasible for the relaxation, while the converse is false: the relaxation optimises over a larger set of solutions. Optimising over a larger set cannot give a worse value, hence:

$$\begin{aligned} \text{maximisation:} \quad & z(LP) \geq z(LP^+) \geq z(MILP); \\ \text{minimisation:} \quad & z(LP) \leq z(LP^+) \leq z(MILP). \end{aligned}$$

In a maximisation the relaxation is an *upper* bound, in a minimisation a *lower* bound. In both cases it is an **optimistic** bound: it promises at least as much as the true optimum, and in general more. It is never a solution, and the value it promises may be unreachable.

The rule is this one, and it holds throughout the course.

- **Primal bounds** come from *feasible integer solutions*: a solution that really exists, built by a heuristic built by hand or found by the solver. They are the pessimistic

side.

- **Dual bounds** come either from the *optimal solution of the LP relaxation* or from any *feasible solution of its dual*. They are the optimistic side.

So the dual of the LP relaxation does *not* give a bound "from the other side": by weak duality, in a minimisation every feasible dual solution is worth at most $z(LP)$, hence at most $z(MILP)$, and it sits on the *same* side as the relaxation — in general weaker. In summary: let $(\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_n)$ be a feasible solution of the MILP, of value $\sum_{j=1}^{n} c_j \bar{x}_j$, and $(\bar{\pi}_1, \bar{\pi}_2, \ldots, \bar{\pi}_m)$ a feasible solution of the dual of the relaxation, of value $\sum_{i=1}^{m} b_i \bar{\pi}_i$. In a **minimisation** problem the first gives $UB(\bar{x})$ and the second $LB(\bar{\pi})$:

$$LB(\bar{\pi}) \;\le\; z(D(LP)) = z(LP) \;\le\; z(LP^{+}) \;\le\; z(MILP) \;\le\; UB(\bar{x}),$$

in a **maximisation** problem the roles are swapped, the first gives $LB(\bar{x})$ and the second $UB(\bar{\pi})$:

$$LB(\bar{x}) \;\le\; z(MILP) \;\le\; z(LP^{+}) \;\le\; z(LP) = z(D(LP)) \;\le\; UB(\bar{\pi}).$$

The equality in the middle is strong duality: the dual of the relaxation is worth exactly the relaxation, and the real inequality is the one between the *value of a feasible dual solution* and that common value.

The two bounds meet in the *sandwich* $LB \le z(MILP) \le UB$ that runs through the whole course: a feasible solution gives the pessimistic side, a relaxation or a solution of its dual gives the optimistic one.

Three "gaps" not to be confused

The word *gap* denotes three different quantities. Let us fix that the denominator is always $|z(MILP)|$ or $|UB|$ as indicated, that the numerator is taken in absolute value and that, when the denominator is zero, the relative gap is not written at all (the absolute difference is reported instead).

1. **Heuristic gap.** If $\bar{x}$ is the solution built by the heuristic, its value $\sum_{j=1}^{n} c_j \bar{x}_j$ is the bound $UB(\bar{x})$ in a minimisation (and $LB(\bar{x})$ in a maximisation), and the gap is

$$\frac{\big|UB(\bar{x}) - z(MILP)\big|}{|z(MILP)|}$$

   when the optimum is known. It is the gap reported in the exercise tables, because the course instances are solved to optimality. When the optimum is *not* known it is computed against a dual bound, which takes the place of $z(MILP)$: the number one gets is larger, and it is the only one that can be claimed.

2. **Certified gap** between two known bounds, without knowing $z(MILP)$: $(UB - LB)/|UB|$ for a minimisation with $UB > 0$. It is what one can claim when optimality has not been proved: it guarantees that $z(MILP)$ lies in the interval $[LB, UB]$, not that it is close to either end.

3. **The solver's** `MIPGap`: the same idea, computed by Gurobi from its own UB (`ObjVal`

of the incumbent) and LB (`ObjBound`), with its convention $|UB - LB|/|UB|$ and its tolerances. It is 0 when optimality has been proved.

The formulas in items 2 and 3 require the incumbent to exist and the denominator to be non-zero: they are conventions, not mathematical identities, and must be stated every time a number is reported.

What the two bounds are actually for

In this course's exercises the instances are small on purpose: Gurobi finds $z(MILP)$ in a fraction of a second, and LB and UB serve to *understand* where a bound comes from and to check the reasoning behind the heuristic and the dual. In practice, however, a real MILP can have thousands of variables and the solver may not terminate in useful time: it is stopped, and what is left is the best solution found so far and the best bound proved at the time of the stop. The certified gap between the two is all one can say with certainty about the distance from the optimum, *without* knowing $z(MILP)$. Being able to build a feasible solution quickly, and to build an optimistic bound by hand, is what remains when the exact optimum is out of reach, and it is also what makes it possible to judge the numbers the solver reports.

1.4 Three patterns that keep coming back

Many MILP constraints with binary selection variables (indexed by a set I) belong to one of three families, depending on the direction:

Name	Constraint	Meaning
set partitioning	$\sum_{i \in I} x_i = 1$	exactly one element of I is chosen
set packing	$\sum_{i \in I} x_i \leq 1$	at most one element of I is chosen
set covering	$\sum_{i \in I} x_i \geq 1$	at least one element of I is chosen

A constraint "every job to exactly one machine" is a *set partitioning*; "at most one job per time slot" is a *set packing*; "every zone covered by at least one station" is a *set covering*, which is the direct translation of an OR clause. Recognising the pattern immediately tells how the constraint behaves: a *covering* allows several elements to be chosen together, a *packing* does not, a *partitioning* imposes both at once.

1.5 What the solver does with the two bounds

A MILP with a bounded feasible set is solved with dedicated algorithms. How they work — how the solution space is explored, how cuts are separated, what to branch on — is not a topic of this course: solution techniques are the subject of a separate course. Only one thing matters here, and it is worth saying at once: the two bounds this course teaches how to build by hand are the two sides of the same sandwich the solver tries to close. They are not handed to it: the solver builds its own, with its own heuristics and relaxations.

- The **primal bound** is a feasible solution, and it gives a value to beat: any part of the space that cannot do better is discarded without being explored.

- The **dual bound** comes from the relaxation, and says how much one may hope for at most: the closer it is to the integer optimum — that is, the tighter the formulation — the less work is left.

The gap between the two is what the solver reports as `MIPGap`, and it is also the only thing one can claim with certainty when the optimum is not reached. Writing the model with a tight formulation and being able to produce the two bounds by hand is what, of this course, is of use when the time comes to solve.

1.6 What this chapter has put in your hands

A MIP model is always written with the same five objects: the **data**, which are known and not decided; the **variables**, one per decision, with their domain; the **objective function**, which says what is maximised or minimised; the **constraints**, one for every sentence of the statement; and the **domain** of the variables, which is a constraint too, and the easiest to forget.

On a written model one then distinguishes a **feasible solution**, which satisfies every constraint, and the **optimal solution**, which among the feasible ones has the best value. Relaxing integrality gives the **linear relaxation**, an easier problem whose optimum is a **bound**: pessimistic if it comes from a feasible solution, optimistic if it comes from a relaxation. The distance between the two is the **gap**, and the rest of the course is about building the two sides and tightening them.

Chapter 2

Relaxations, duality and bounds

Script: `python/cap04_bounds.py` Online page

This chapter teaches how to produce, *by hand*, a number that certainly lies on one side of the integer optimum. It serves three purposes: understanding how good a model is, understanding how good a heuristic is, and reading with judgement the numbers a solver reports when it has not finished.

2.1 What a relaxation is

Operational definition

A **relaxation** of a minimisation problem with feasible set X is a problem with the same objective and a larger feasible set, $X \subseteq \hat{X}$: the only thing relaxed are the constraints. Then

$$\min_{x \in \hat{X}} \sum_{j=1}^{n} c_j x_j \;\leq\; \min_{x \in X} \sum_{j=1}^{n} c_j x_j,$$

because the minimum over a larger set cannot be higher. In a maximisation the inequality is reversed.

The three relaxations used in this course, in increasing order of strength:

Name	What is dropped	Note
$z(LP)$, relaxation without the bounds	$x \in \{0,1\}$ becomes $x \geq 0$	this is the one whose dual is written by hand: it has fewer constraints and so a dual with fewer variables
$z(LP^{+})$, with the bounds	$x \in \{0,1\}$ becomes $0 \leq x \leq 1$	this is Gurobi's `relax()`, the first relaxation the solver solves
$z(LP^{++})$, with the bounds and the valid inequalities	as above, plus valid inequalities	Section 2.7 explains which ones

In a minimisation, $z(LP) \leq z(LP^{+}) \leq z(LP^{++}) \leq z(MILP)$.

If the model contains an assignment constraint $\sum_m x_{jm} = 1$ with $x \geq 0$, then $x_{jm} \leq 1$ is already implied and the two relaxations are *equal*. It happens whenever an equality constraint already forces the variables below one, which is the case of every assignment model. When the two do differ, the difference must be reported: it is information about the model.

2.2 The primal/dual conversion table

From a minimisation primal to its dual

Let the primal be a minimisation, with m constraints indexed by i and n variables indexed by j.

In the primal (min)	In the dual (max)
constraint i of type $\geq$	variable $\pi_i \geq 0$
constraint i of type $\leq$	variable $\pi_i \leq 0$
equality constraint i	free variable π_i
variable $x_j \geq 0$	constraint j of type $\leq c_j$
free variable x_j	equality constraint j, $= c_j$

The dual objective is $\max \sum_i b_i \pi_i$ and dual constraint j is $\sum_i a_{ij} \pi_i \lesseqgtr c_j$ with the direction given by the table. If the primal is a **maximisation**, every direction in the table is reversed and the dual is a minimisation.

How to read the dual

Dual constraint j says: "the value I attach to the resources consumed by activity j cannot exceed its cost". With this reading, every recipe for building a feasible dual solution has an economic meaning, and that is what makes it memorable: Section 2.4 gives three.

2.3 Weak duality, strong duality

The two theorems, and what they are for here

Let $(\bar{x}_1, \ldots, \bar{x}_n)$ be a feasible solution of a minimisation model of LP type and $(\bar{\pi}_1, \ldots, \bar{\pi}_m)$ a feasible solution of its dual.

- **Weak duality**: $\sum_{i=1}^{m} b_i \, \bar{\pi}_i \leq \sum_{j=1}^{n} c_j \, \bar{x}_j$. *Always*, with no assumptions. This is the one we need: it gives a lower bound from *any* feasible dual solution, even one built by hand and far from optimal.
- **Strong duality**: if the LP has a finite optimum, then $z(D(LP)) = z(LP)$. It serves as a *check*: the optimum of the dual written by hand must coincide with $z(LP)$; if it does not, the dual is written wrong. The course scripts verify it with an `assert` in `due_rilassamenti`.

And then the step to the integer problem, which is trivial but must be said: since every

feasible solution of the MILP is feasible for the relaxation too,

$$\sum_{i=1}^{m} b_i \bar{\pi}_i \;\leq\; z(LP) \;\leq\; z(MILP).$$

A dual bound of the LP is a bound for the MILP too, on the same side.

The dual one writes is that of the *relaxation*, not of the MILP. A MILP has no linear dual, and strong duality between a MILP and any linear program does not hold in general: the jump $z(MILP) - z(LP)$ is precisely what is missing. When this course says "the dual", it always means the dual of the stated relaxation.

2.4 Three recipes for building a dual solution by hand

The recipes

1. **Zero out and saturate.** Set all dual variables to zero except one family, and push those to the largest value (in a maximisation dual) compatible with the dual constraints. On an assignment, for instance, one sets $\bar{\pi} = 0$ and obtains $\bar{\mu}_j = \min_m c_{jm}$, that is "every job costs at least its cheapest option".

2. **Constructive heuristic on the constraints.** Scan the primal constraints one at a time, raise the corresponding dual variable until the first dual constraint that opposes it becomes tight, and update the residuals. This is the recipe used below for the covering problem.

3. **The best ratio.** With a single capacity constraint in a maximisation, the dual variable $\bar{v} = \max_j p_j/w_j$ is feasible and gives the bound $b\bar{v}$: "all the capacity used at the best efficiency".

Whichever recipe is used, the constructed solution must be *checked* feasible for the dual — that is the only thing that makes the bound valid — and its value compared with $z(LP)$ to see how much was left on the table.

2.5 A minimisation problem, in full

Minimum-cost zone covering

A town has $n \in \mathbb{Z}_{\geq 1}$ districts; each has an emergency team, and activating the team of district $j \in \{1, 2, \ldots, n\}$ costs $c_j \in \mathbb{Q}_{>0}$ thousand euros. There are $m \in \mathbb{Z}_{\geq 1}$ sensitive zones: zone $i \in \{1, 2, \ldots, m\}$ is covered if at least one of the teams in the set S_i of the districts bordering it is active. All zones must be covered at minimum cost.

Data: $n = 4$ districts of cost $c = (4, 3, 5, 3)$, and $m = 6$ zones, which correspond to the six pairs of districts, in the order $\{1, 2\}$, $\{2, 3\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 4\}$, $\{3, 4\}$. With $x_j = 1$ if team j is active (n binary variables) and S_i the set of teams covering zone i:

Minimum-cost covering

$$\min \sum_{j=1}^{n} c_j\, x_j \tag{2.1a}$$

$$\text{subject to} \sum_{j \in S_i} x_j \geq 1, \qquad \forall i \in \{1, 2, \ldots, m\}, \tag{2.1b}$$

$$x_j \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{2.1c}$$

It is a *set covering*: m covering constraints, one per zone.

The dual of the relaxation without the bounds. Relaxing (2.1c) to $x_j \geq 0$ and attaching to each constraint (2.1b) a dual variable $\pi_i \geq 0$ ($\geq$ direction in a minimisation, by the conversion table):

Dual of the LP relaxation

$$\max \sum_{i=1}^{m} \pi_i \tag{2.2a}$$

$$\text{subject to} \sum_{i\,:\,j \in S_i} \pi_i \leq c_j, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{2.2b}$$

$$\pi_i \geq 0, \qquad \forall i \in \{1, 2, \ldots, m\}. \tag{2.2c}$$

Description. The variable π_i is the price attached to zone i. The objective (2.2a) adds up the prices of the m zones. Constraints (2.2b) are the columns of the x_j, one per team: the zones team j covers, priced at those values, cannot be worth more than what it costs to activate it, namely c_j. This is the economic reading of the previous section: no team may turn out to be a bargain.

For the instance, every team covers three zones and the four dual constraints are

$$\pi_1 + \pi_3 + \pi_4 \leq 4, \quad \pi_1 + \pi_2 + \pi_5 \leq 3, \quad \pi_2 + \pi_3 + \pi_6 \leq 5, \quad \pi_4 + \pi_5 + \pi_6 \leq 3.$$

Covering on the instance — primal

$$\begin{aligned}
\min\ & 4x_1 + 3x_2 + 5x_3 + 3x_4 \\
& x_1 + x_2 \geq 1 \\
& x_2 + x_3 \geq 1 \\
& x_1 + x_3 \geq 1 \\
& x_1 + x_4 \geq 1 \\
& x_2 + x_4 \geq 1 \\
& x_3 + x_4 \geq 1 \\
& x_1, x_2, x_3, x_4 \in \{0, 1\}
\end{aligned}$$

Covering on the instance — dual of the LP relaxation

$$
\begin{aligned}
\max\ & \pi_1 + \pi_2 + \pi_3 + \pi_4 + \pi_5 + \pi_6 \\
& \pi_1 \qquad + \pi_3 + \pi_4 \qquad\qquad \le 4 \\
& \pi_1 + \pi_2 \qquad\qquad + \pi_5 \qquad \le 3 \\
& \qquad \pi_2 + \pi_3 \qquad\qquad + \pi_6 \le 5 \\
& \qquad\qquad\qquad \pi_4 + \pi_5 + \pi_6 \le 3 \\
& \pi_1,\ \pi_2,\ \pi_3,\ \pi_4,\ \pi_5,\ \pi_6 \ge 0
\end{aligned}
$$

A dual solution by hand (recipe 2). Scan the zones in order; for zone i raise π_i by the most the residuals of the dual constraints of the teams in S_i allow, then update the residuals.

- **Zone 1** ($S_1 = \{1, 2\}$): residuals $(4, 3, 5, 3)$, the smallest among teams 1 and 2 is 3. Set $\bar{\pi}_1 = 3$; residuals $(1, 0, 5, 3)$.
- **Zone 2** ($\{2, 3\}$): the residual of team 2 is 0, so $\bar{\pi}_2 = 0$.
- **Zone 3** ($\{1, 3\}$): the smallest of 1 and 5 is 1. Set $\bar{\pi}_3 = 1$; residuals $(0, 0, 4, 3)$.
- **Zones 4 and 5** ($\{1, 4\}$ and $\{2, 4\}$): teams 1 and 2 have zero residual, so $\bar{\pi}_4 = \bar{\pi}_5 = 0$.
- **Zone 6** ($\{3, 4\}$): the smallest of 4 and 3 is 3. Set $\bar{\pi}_6 = 3$; residuals $(0, 0, 1, 0)$.

All dual constraints hold by construction, and the value is

$$LB \;=\; \bar{\pi}_1 + \cdots + \bar{\pi}_6 \;=\; 3 + 0 + 1 + 0 + 0 + 3 \;=\; 7.$$

A primal upper bound. The covering constructive heuristic picks, at each step, the team with the best ratio between cost and zones still uncovered: it takes teams 1, 2 and 4, of cost $4 + 3 + 3 = 10$. The solution is feasible and *integer*, so $UB = 10$.

The sandwich, and what the solver says. $z(LP) = z(LP^+) = 15/2$ (the two relaxations coincide, and the optimum of the dual confirms it: strong duality) and $z(MILP) = 10$, attained with teams $1, 2, 4$ — the same solution as the heuristic, which here is optimal.

UB (constructive heuristic)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
10	7	15/2	10	0.0%

Read it this way: the hand-built bound is 7, the relaxation 7.5, the optimum 10. The certified gap between our two hand-built bounds is $(10 - 7)/10 = 30\%$: without solving the MILP we would know only that the optimum lies between 7 and 10. The heuristic was already optimal, but the bounds cannot tell us that.

2.6 A maximisation problem: the roles swap

Knapsack with several resources

There are $n \in \mathbb{Z}_{\geq 1}$ items and $m \in \mathbb{Z}_{\geq 1}$ resources: item $j \in \{1, 2, \ldots, n\}$ has value $p_j \in \mathbb{Q}_{>0}$ and uses $a_{ij} \in \mathbb{Q}_{\geq 0}$ units of resource $i \in \{1, 2, \ldots, m\}$, of which $b_i \in \mathbb{Q}_{>0}$ units are available. Choose the subset of maximum value that fits inside *every* availability.

The data of the instance are $n = 5$ jobs of value $p = (10, 9, 7, 6, 4)$ and $m = 3$ resources — steel, lathe hours and testing hours — of availability $b = (10, 9, 8)$. The consumptions are

$$a = \begin{pmatrix} 6 & 4 & 3 & 3 & 2 \\ 1 & 3 & 4 & 2 & 3 \\ 3 & 2 & 2 & 4 & 1 \end{pmatrix},$$

one row per resource and one column per job.

The following n binary variables are introduced:

$$x_j = \begin{cases} 1 & \text{if job } j \text{ is chosen,} \\ 0 & \text{otherwise,} \end{cases} \qquad \forall j \in \{1, 2, \ldots, n\}.$$

Knapsack with several resources

$$\max \quad \sum_{j=1}^{n} p_j \, x_j \tag{2.3a}$$

$$\text{subject to} \quad \sum_{j=1}^{n} a_{ij} \, x_j \leq b_i, \qquad \forall i \in \{1, 2, \ldots, m\}, \tag{2.3b}$$

$$x_j \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}. \tag{2.3c}$$

Description. The objective (2.3a) sums the value of the chosen jobs. The constraints (2.3b) are the capacities: one per resource, and each one says that the total consumption of the chosen jobs does not exceed the availability. The domains (2.3c) are the n binarity conditions, one per job: a job is either done in full or not at all.

The dual of the relaxation without the bounds. Relax the domain (2.3c) to $x_j \geq 0$. The primal is a maximisation with m constraints, all of direction $\leq$: the conversion table gives m dual variables $v_i \geq 0$, one per resource, and one dual constraint for each of the n primal variables.

Dual of the LP relaxation of the knapsack model

$$\min \quad \sum_{i=1}^{m} b_i\, v_i \tag{2.4a}$$

$$\text{subject to} \quad \sum_{i=1}^{m} a_{ij}\, v_i \geq p_j, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{2.4b}$$

$$v_i \geq 0, \qquad \forall i \in \{1, 2, \ldots, m\}. \tag{2.4c}$$

Description. The variable v_i is the price of one unit of resource i. The objective (2.4a) is the value attached to all the availabilities. The n constraints (2.4b), one per job, say that the resources used by job j, valued at those prices, are worth no less than the profit that job brings. Every dual constraint now has m terms, one per resource: the column of the variable x_j in the primal has m rows.

Written out in full on the data of the instance, the primal and the dual are the following.

Knapsack on the instance — primal

$$\begin{aligned} \max\ & 10x_1 + 9x_2 + 7x_3 + 6x_4 + 4x_5 \\ \text{subject to}\ & 6x_1 + 4x_2 + 3x_3 + 3x_4 + 2x_5 \leq 10 \\ & x_1 + 3x_2 + 4x_3 + 2x_4 + 3x_5 \leq 9 \\ & 3x_1 + 2x_2 + 2x_3 + 4x_4 + x_5 \leq 8 \\ & x_1,\ x_2,\ x_3,\ x_4,\ x_5 \in \{0, 1\} \end{aligned}$$

Knapsack on the instance — dual of the LP relaxation

$$\begin{aligned} \min\ & 10v_1 + 9v_2 + 8v_3 \\ \text{subject to}\ & 6v_1 + v_2 + 3v_3 \geq 10 \\ & 4v_1 + 3v_2 + 2v_3 \geq 9 \\ & 3v_1 + 4v_2 + 2v_3 \geq 7 \\ & 3v_1 + 2v_2 + 4v_3 \geq 6 \\ & 2v_1 + 3v_2 + v_3 \geq 4 \\ & v_1,\ v_2,\ v_3 \geq 0 \end{aligned}$$

The two bounds.

- **Heuristic** (constructive heuristic by ratio of value to total consumption): the ratios are $10/10 = 1$, $9/9 = 1$, $7/9$, $6/9$ and $4/6$; job 1 is taken and then job 2, and nothing else fits because the steel has run out. Value $10 + 9 = 19$. In a **maximisation** the heuristic gives a **lower** bound: $LB = 19 \leq z(MILP)$.
- **Dual by hand** (recipe 3, adapted): one pays *a single* resource. Choosing resource i and setting $\bar{v}_i = \max_j p_j / a_{ij}$ with all the others at zero, every dual constraint is satisfied by the single term $a_{ij}\bar{v}_i \geq p_j$. The three choices give $10 \cdot \frac{7}{3} = \frac{70}{3}$ with the steel, $9 \cdot 10 = 90$ with the lathe and $8 \cdot \frac{9}{2} = 36$ with the testing: the cheapest is the steel. In a **maximisation** the dual gives an **upper** bound: $z(MILP) \leq UB = \frac{70}{3} \approx 23.33$.

The sandwich. $z(MILP) = 22$ (jobs 2, 3 and 4, which saturate all three resources), $z(LP) = 159/7 \approx 22.71$ and $z(LP^+) = 22$:

$$19 \;\le\; z(MILP) = 22 \;=\; z(LP^+) \;\le\; z(LP) = \tfrac{159}{7} \;\le\; UB = \tfrac{70}{3}.$$

LB (constructive heuristic)	$z(MILP)$	$z(LP^+)$	$z(LP)$	UB (dual by hand)	heuristic gap
19	22	22	159/7	70/3	13.6%

Here the dual by hand is *not* optimal for the relaxation ($70/3 > 159/7$): paying a single resource is a valid recipe, but a coarse one, because it ignores that the other two are saturated as well. The relaxation with the bounds is worth 22 and coincides with the integer optimum: the constraint $x_j \le 1$ bites, because without it the LP takes more than one unit of the best job.

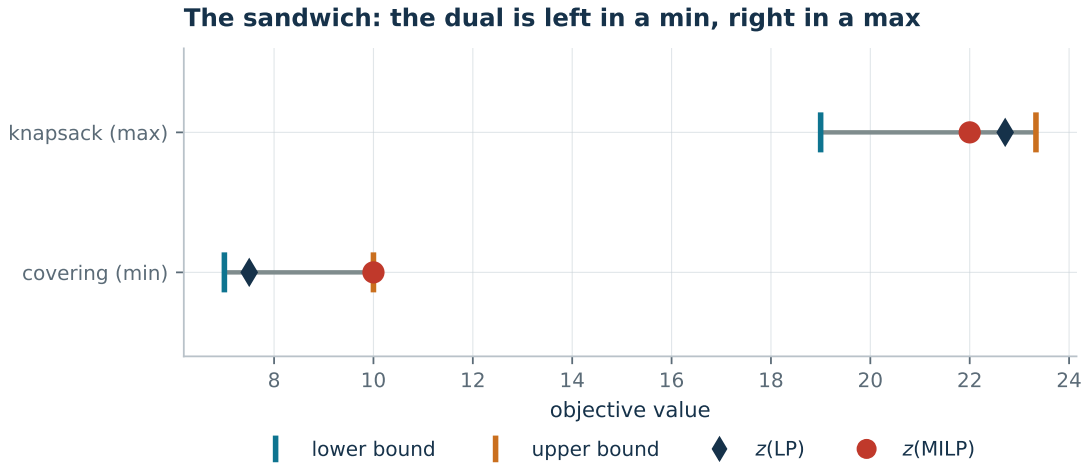


Figure 2.1: The two examples. In a minimisation the dual bound lies to the left of the optimum and the heuristic to the right; in a maximisation the roles swap.

The sandwich, written once and for all

minimisation: $LB(\bar\pi) \;\le\; z(D(LP)) = z(LP) \;\le\; z(LP^+) \;\le\; z(MILP) \;\le\; UB(\bar x)$

maximisation: $LB(\bar x) \;\le\; z(MILP) \;\le\; z(LP^+) \;\le\; z(LP) = z(D(LP)) \;\le\; UB(\bar\pi)$

where $(\bar\pi_1, \bar\pi_2, \dots, \bar\pi_m)$ is a feasible dual solution of the relaxation, of value $\sum_{i=1}^{m} b_i\,\bar\pi_i$, and $(\bar x_1, \bar x_2, \dots, \bar x_n)$ a feasible solution of the MILP, of value $\sum_{j=1}^{n} c_j\,\bar x_j$. The chains read as follows: the *relaxation side* is optimistic and holds all the dual bounds; the *heuristic side* is pessimistic and holds all the feasible solutions. The name (LB or UB) depends on the direction of the objective, the role does not.

2.7 Valid inequalities and constraints that preserve optimality

Two different notions

- A **valid inequality** for a MILP is satisfied by *all* feasible integer solutions. Adding it does not change $z(MILP)$; it may reduce $z(LP^+)$, and then it is called a **cut**.
- A **constraint that preserves optimality** cuts off some feasible solutions, but not all the optimal ones: after adding it an optimum still exists and $z(MILP)$ does not change. It is not a valid inequality, and it must be declared as such. Example: let z be a variable the objective minimises, appearing only in constraints $z \geq t_j x_j$. Adding $z \leq My$, with y an activation binary, cuts off feasible solutions — those with $y = 0$ and z positive — but not the optimal ones, because at the optimum z is already pushed down by the objective.

The difference matters: the first can always be added; the second only after proving that at least one optimum survives, and never if one then wants to enumerate *all* the optimal solutions.

The cover cut for the knapsack

A set S of items is a **cover** if $\sum_{j \in S} w_j > b$: they do not all fit together. Then

$$\sum_{j \in S} x_j \;\leq\; |S| - 1$$

is a valid inequality. The cover is **minimal** if no proper subset of it is already a cover; the cuts of minimal covers are the strongest.

On the knapsack instance ($w = (5, 4, 3, 3)$, $b = 9$) the minimal covers are the four triples $\{1,2,3\}$, $\{1,2,4\}$, $\{1,3,4\}$, $\{2,3,4\}$, of weight 12, 12, 11, 10: all > 9, and no pair is. The four cuts are $\sum_{j \in S} x_j \leq 2$.

The optimal solution of the relaxation $z(LP^+) = 71/4$ is $\tilde{x} = (1,\ 1/4,\ 1,\ 0)$. On it:

Cover S	$\sum_{j \in S} \tilde{x}_j$	$\lvert S \rvert - 1$	
$\{1,2,3\}$	$9/4$	2	**violated**: the cut is needed
$\{1,2,4\}$	$5/4$	2	satisfied
$\{1,3,4\}$	2	2	satisfied (with equality)
$\{2,3,4\}$	$5/4$	2	satisfied

Adding all four cuts, $z(LP^+)$ drops from $71/4 = 17.75$ to $69/4 = 17.25$, and $z(MILP)$ stays 17: the relaxation has moved closer to the integer optimum without changing it. This is the mechanism by which a modern solver closes gaps.

2.8 Stronger formulations: what it means, and how it is proved

The criterion

Two formulations A and B of the same problem are compared in two steps.

1. **Same integer set**: $X_A \cap \mathbb{Z}^n = X_B \cap \mathbb{Z}^n$. Without this step nothing is being compared. It is proved in both directions, using binarity or integrality where needed.
2. **Relaxation**: B is *stronger* than A if $X_B \subseteq X_A$ as polyhedra, and then $z_B(LP^+)$ is at least as good as $z_A(LP^+)$. If the inclusion is strict *and* one exhibits a point of $X_A \setminus X_B$ that is optimal for the relaxation of A, the improvement is proved on that instance.

The reference case is the activation of Section 6.1: $x_{ij} \leq y_j$ implies $\sum_i x_{ij} \leq n y_j$, so the disaggregated form is stronger; on three customers and two sites the relaxation goes from 38/3 to 15, which is already the integer optimum.

A stronger formulation has fewer nodes but more rows, and every node costs more. The balance depends on the instance. What can be *proved* is the strength of the relaxation; speed is *measured*.

2.9 What the solver says

The four attributes to read, and in which order

1. `Status`: 2 is `OPTIMAL`, 3 `INFEASIBLE`, 9 `TIME_LIMIT`. To be read *before* anything else.
2. `SolCount`: how many integer solutions were found. If it is 0, `ObjVal` does not exist and reading it is an error.
3. `ObjVal` and `ObjBound`: the best solution found and the best bound proved. In a minimisation, `ObjBound` $\leq z(MILP) \leq$ `ObjVal`.
4. `MIPGap` and `NodeCount`: the certified gap between the two and how many nodes were explored.

On the covering instance, the relaxation of the model *as we wrote it* is 15/2 and the integer optimum is 10. Yet Gurobi reports `ObjBound` = 10 and `NodeCount` = 0: it closed the gap at the root, with presolve, its own cuts and heuristics, without ever branching. Switching those off (`Presolve = Cuts = Heuristics = 0`) the same model gives the same optimum 10 but with 5 nodes. Two consequences:

- "how hard a model is" is not a property of the model alone: it also depends on what the solver brings;
- the relaxation we speak of in the hand-built bounds is that of the model as written,

and it is obtained with `relax()`: reading `ObjBound` at the end of the solve gives the optimum, not the relaxation.

2.10 LP duals are not the marginal prices of the MILP

The claim, and the counterexample

In an LP, the dual $\tilde{\pi}_i$ of a resource constraint is its *marginal price*: increasing b_i by a small amount changes the optimum by $\tilde{\pi}_i$ per unit (as long as the basis does not change). **For a MILP this is false**, and not by approximation.
On the knapsack, as the capacity b varies:

b	$z(MILP)$	$z(LP^+)$	LP dual	true change in $z(MILP)$
8	16	16	2	—
9	17	71/4	7/4	+1
10	17	39/2	7/4	**0**
11	20	85/4	7/4	+3
12	23	23	7/4	+3

The LP dual is the ratio p_j/w_j of the "critical" item: it is 2 when the capacity runs out on item 1 and 7/4 when there is room left for item 2. The true change of the integer optimum comes in jumps: from $b = 9$ to $b = 10$ it does not change *at all* (no subset exploits the extra unit), while the dual promises 7/4; from 10 to 11 it jumps by 3.

Of the LP dual, the only use this course makes of it remains true: it is a **bound**. $\tilde{\pi}_i$ says how much the resource is worth *in the relaxation*, and the sum $b'\tilde{u}$ is a valid bound for the MILP. As managerial advice ("is it worth buying one more unit?") it must instead be checked by solving the MILP again: the difference $z(MILP)(b_i + 1) - z(MILP)(b_i)$ is the only correct answer, and there is no closed formula for it.

2.11 The bound protocol

Faced with any model, the route to tightening the sandwich is always the same, and it pays to follow it in order:

1. a **feasible and integer solution**, built by hand, checked against every constraint *and* against integrality — it is the integrality check that makes that value a bound of the MILP and not of the relaxation alone;
2. the **dual of the relaxation without the bounds**, written in general form and then on the instance, with the directions taken from the conversion table;
3. a **feasible dual solution built by hand**, with the recipe stated, checked feasible and evaluated;
4. the two **relaxations** $z(LP)$ and $z(LP^+)$, noting whether they coincide and why;

5. the **optimum** $z(MILP)$ and the table $UB \cdot LB \cdot z(LP) \cdot z(LP^+) \cdot z(MILP)$ with the gap, defined as in Chapter 1;

6. the **additional remarks**: valid inequalities with their count, constraints preserving optimality declared as such, comparison of the relaxations of alternative formulations.

Steps 1 and 3 are the two sides of the sandwich, and the only ones that can be built without a solver. Steps 4 and 5 measure how good the first two were.

Chapter 3

From the model to Python/Gurobi

Script: `python/cap06_gurobi.py` Online page

The course uses **one solver only**, Gurobi from Python. This chapter shows how a model is written — one family of constraints per block, with the names of the mathematical model — and above all how the results are *read*, including the case where the solver has not finished.

3.1 The basic gurobipy instructions

A model is written in gurobipy with a handful of instructions, always the same ones, and from here on the line of code that writes it appears next to every model. It pays to see them once on a small, complete model: the **binary knapsack**, where out of n items of value p_j and weight w_j one chooses a subset of weight at most c worth as much as possible.

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} p_j\, x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} w_j\, x_j \le c, \\ & x_j \in \{0,1\}, \qquad \forall j \in \{1,2,\dots,n\}. \end{aligned}$$

```
import gurobipy as gp                          # the module, always abbreviated gp
from gurobipy import GRB                       # the constants: GRB.BINARY, GRB.MAXIMIZE...

p = [10, 7, 6, 4]                              # values
w = [5, 4, 3, 3]                               # weights
c = 9                                          # capacity
n = len(p)

m = gp.Model("knapsack")                                          # an empty model
x = m.addVars(n, vtype=GRB.BINARY, name="x")                      # n binary variables
m.setObjective(gp.quicksum(p[j] * x[j] for j in range(n)), GRB.MAXIMIZE)
```

```
m.addConstr(gp.quicksum(w[j] * x[j] for j in range(n)) <= c, name="
    capacity")
m.optimize()
```

The instructions, one per line of the model:

- `Model` creates the model; everything else hangs off it.
- `addVars` adds a whole indexed family: `addVars(n)` gives $x_0, \ldots, x_{n-1}$, while `addVars(n, k)` gives the matrix x_{ij}, indexed as `x[j]` and `x[i, j]`. For a single variable there is `addVar`. Python indices start at zero, written models start at one: that is the only difference to keep in mind.
- `vtype` declares the domain: `GRB.BINARY` for $x \in \{0, 1\}$, `GRB.INTEGER` for $x \in \mathbb{Z}_{\geq 0}$, and without `vtype` the variable is continuous and non-negative. The domain is a constraint: declaring it wrongly is declaring a different model.
- `quicksum` builds a sum $\sum_j c_j x_j$; it does what `sum` does but without building intermediate expressions, and it is the one always used.
- `setObjective` takes the expression and the direction, `GRB.MINIMIZE` or `GRB.MAXIMIZE`.
- `addConstr` adds *one* constraint, `addConstrs` a whole family, and wants a generator in round brackets: one line of code per family of constraints, just as it is one line in the written model.
- `optimize` solves.

The `name` is of no use to the solver: it is of use to whoever reads the exported model or an error message, and it is worth always matching it to the name of the constraint family in the written model.

With the same instructions one writes the other classical problems that come back in these pages. **Bin packing** — n items of size w_j into bins of capacity c, using as few as possible — wants two families of binaries, x_{ji} for "item j goes into bin i" and y_i for "bin i is used", and two families of constraints:

```
x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
y = m.addVars(k, vtype=GRB.BINARY, name="y")
m.setObjective(y.sum(), GRB.MINIMIZE)
m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
m.addConstrs((gp.quicksum(w[j] * x[j, i] for j in range(n)) <= c * y[i]
              for i in range(k)), name="capacity")
```

Scheduling on identical machines ($P||z$) changes only the objective and one constraint: a continuous T for the makespan, $\min T$, and $\sum_j t_j x_{ji} \leq T$ for every machine. The **travelling salesman** has one binary per arc, x_{ij}, and two families of equalities — every city is left once and entered once — plus the constraints forbidding subtours, which are the interesting part and are not needed here.

3.2 The four classes of variables

What	vtype	Domain	Typical use
yes/no decision	`GRB.BINARY`	$\{0, 1\}$	selection, activation, assignment
count	`GRB.INTEGER`	$\mathbb{Z}$ between `lb` and `ub`	boxes, shifts, workers
measurable quantity	(default)	$[\texttt{lb}, \texttt{ub}] \subseteq \mathbb{R}$	time, money, flow
free variable	(default) with `lb=-GRB.INFINITY`	$\mathbb{R}$	duals of equalities

`GRB.BINARY` already implies `lb` $= 0$ and `ub` $= 1$: there is no need to repeat them. A continuous variable has `lb = 0` by default: a variable that must be allowed to go negative — typically the dual of an equality, or a signed deviation — must be declared with `lb=-GRB.INFINITY`, otherwise the model is silently wrong.

3.3 The model, one family per block

```
def model(t, c, a):
    """Problem 7.1: one addConstrs per family, with the label as its
       name."""
    m = gp.Model("assignment");  m.Params.OutputFlag = 0
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")            # data ->
        variables
    m.setObjective(gp.quicksum(c[j][h] * x[j, h] for j in range(n)
                               for h in range(k)), GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
    m.addConstrs((gp.quicksum(t[j][h] * x[j, h] for j in range(n)) <= a
       [h]
                  for h in range(k)), name="availability")
    return m, x
```

The three writing rules of the course

1. **One `addConstrs` per family**, in the order of the mathematical model, and the `name` is the label of the family (`"assign"`, `"availability"`). The generated names are `assign[0]`, `assign[1]`, ...: they reappear in the duals and in the solver messages.
2. **Variables are declared all at once**, with `addVars` and the indices of the model. `x.sum(j, "*")` is the sum over all the indices of the second dimension: it is the writing of the model's *dummy index*.
3. **Data come in as arguments**, not as globals: the same function serves the base instance and all the variants of the modelling questions.

The model as written has 9 variables, 6 constraints and 18 nonzero coefficients. To check that it is what was written on paper, `m.write("model.lp")` produces the readable LP format:

```
Minimize
  5 x[0,0] + 10 x[0,1] + 2 x[0,2] + 5 x[1,0] + 4 x[1,1] + 6 x[1,2]
   + 5 x[2,0] + 4 x[2,1] + 6 x[2,2]
Subject To
 assign[0]: x[0,0] + x[0,1] + x[0,2] = 1
 assign[1]: x[1,0] + x[1,1] + x[1,2] = 1
 assign[2]: x[2,0] + x[2,1] + x[2,2] = 1
 availability[0]: 2 x[0,0] + 3 x[1,0] + 4 x[2,0] <= 5
 availability[1]: x[0,1] + 4 x[1,1] + 5 x[2,1] <= 6
 availability[2]: 3 x[0,2] + 2 x[1,2] + 3 x[2,2] <= 7
Bounds
Binaries
 x[0,0] x[0,1] x[0,2] x[1,0] x[1,1] x[1,2] x[2,0] x[2,1] x[2,2]
End
```

It is the quickest way to spot a wrong coefficient: the instance table in the notes and this output must agree line by line.

3.4 Reading the results

The reading order, which never changes

1. `Status`. Always first.
2. `SolCount`: if it is 0, *no* solution exists and `ObjVal` cannot be read.
3. `ObjVal` and `ObjBound`: the best solution found and the best bound proved.
4. `MIPGap`, `NodeCount`, `Runtime`: how much remains uncertain, and how much it cost.

Status	value	SolCount	What can be said
OPTIMAL	2	≥ 1	$z(MILP) =$ `ObjVal`, proved
INFEASIBLE	3	0	the model has no feasible solution
UNBOUNDED	5	0	a constraint, or a bound on a variable, is missing
TIME_LIMIT	9	0	nothing: neither a solution nor, in general, a useful bound
TIME_LIMIT	9	≥ 1	`ObjBound` $\leq z(MILP) \leq$ `ObjVal`
SOLUTION_LIMIT	10	≥ 1	as above

The four cases on the assignment instance

- **Normal solve.** `Status` $= 2$, `SolCount` $= 2$, `ObjVal` $=$ `ObjBound` $= 11$, `MIPGap` $= 0$, `NodeCount` $= 0$: Gurobi closed everything at the root.
- **Infeasible.** With availability $a = (1, 1, 1)$ no assignment is possible: `Status` $= 3$, `SolCount` $= 0$.
- **Stopped immediately.** With `TimeLimit = 0`: `Status` $= 9$, `SolCount` $= 0$ and `ObjBound` $= -\infty$. There is nothing to report: neither a solution nor a bound.
- **Stopped at the first solution.** With `SolutionLimit = 1`: `Status` $= 10$,

`SolCount` = 1, `ObjVal` = 12, `ObjBound` = 10, `MIPGap` = 0.1667. This is the case where an *interval* is reported: the optimum lies between 10 and 12, and the 16.7% measures its width. Saying "the optimum is 12" would be false.

One does not write "the optimum is `ObjVal`" if `Status` is not `OPTIMAL`. One does not write a gap if `SolCount` is 0. One does not compare the `Runtime` of two models solved with different settings. And one does not read `ObjBound` at the end of the solve thinking it is the root relaxation: for that, `relax()` is needed, as in Section 2.9.

3.5 Tolerances

Parameter	Default	Meaning
`IntFeasTol`	10^{-5}	how far an integer variable may be from an integer
`FeasibilityTol`	10^{-6}	violation allowed on a linear constraint
`OptimalityTol`	10^{-6}	tolerance on the reduced costs
`MIPGap`	10^{-4}	relative gap below which the solver stops

A binary may come back as 0.9999999997. In the text one writes 1: values are rounded *when they are reported*, and comparisons always use a tolerance — in this course 10^{-6}, the constant `TOL` of `python/mip.py`. Writing `if x.X == 1` is a mistake; one writes `if x.X > 0.5`.
The default `MIPGap` $= 10^{-4}$ also means the solver may stop *before* the exact optimum while declaring `OPTIMAL`: on instances with large values it is worth lowering it.

3.6 The relaxation with `relax()`

```
m.update()              # relax() copies the model: pending changes must
    be applied first
r = m.relax()           # binaries become 0 <= x <= 1, integers x >= lb
r.Params.OutputFlag = 0
r.optimize()
zlp = r.ObjVal
duals = {c.ConstrName: c.Pi for c in r.getConstrs()}
```

On that instance, $z(LP^+) = z(LP) = 53/5$ — the two relaxations coincide, because the assignment constraints already imply $x_{jm} \le 1$ — and the nonzero duals are

$$\tilde{\mu} = (2,\ 4.8,\ 5), \qquad \tilde{\pi}_2 = -0.2,$$

that is, the three dual variables of the `assign` constraints and the one of `availability[1]`: machine 2 is the only tight resource.

The relaxation without the bounds, when it is needed

The function `rilassamento(m, rafforzato=False)` of `python/mip.py` also drops the upper bound of the binaries: it is the relaxation whose dual the notes write by hand. Both versions are reported, with a note on whether they coincide and why.

3.7 Three problems the course reuses

So far the example model has always been the knapsack. The three problems of this section come back in chapter 4, where the solutions of next-fit, first-fit, best-fit, LPT and nearest neighbour are built by hand: here their models are written, so that those heuristics have an optimum to compare themselves with.

Bin packing: how many containers are enough

Script: `python/cap06_bpp.py`

Bin packing

There are $n \in \mathbb{Z}_{\geq 1}$ items: item $j \in \{1, 2, \ldots, n\}$ weighs $w_j \in \mathbb{Q}_{>0}$. The containers are all alike, of capacity $c \in \mathbb{Q}_{>0}$, and at most one per item may be used. Use the smallest number of containers.

The model

Two families of binary variables are needed: $x_{jb} = 1$ if item j goes into container b, and $y_b = 1$ if container b is used.

Bin packing

$$\min \quad \sum_{b=1}^{k} y_b \tag{3.1a}$$

$$\text{subject to} \quad \sum_{b=1}^{k} x_{jb} = 1, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{3.1b}$$

$$\sum_{j=1}^{n} w_j \, x_{jb} - c \, y_b \leq 0, \qquad \forall b \in \{1, 2, \ldots, k\}, \tag{3.1c}$$

$$x_{jb} \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}, \ \forall b \in \{1, 2, \ldots, k\}, \tag{3.1d}$$

$$y_b \in \{0, 1\}, \qquad \forall b \in \{1, 2, \ldots, k\}. \tag{3.1e}$$

Description. The objective (3.1a) counts the containers switched on. The constraints (3.1b) say that every item ends up in exactly one container. The constraints (3.1c) are the capacity written as an *activation*: while $y_b = 0$ container b can receive nothing, and as soon as $y_b = 1$ it takes up to c. The domains (3.1d) and (3.1e) are the two families of binaries, one per item–container pair and one per container.

Building it in gurobipy

```
def modello_bpp(w, c, k):
    n = len(w)
    m = nuovo_modello("bin_packing")
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
    y = m.addVars(k, vtype=GRB.BINARY, name="y")
    m.setObjective(y.sum(), GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in R(n)), name="item")
    m.addConstrs((gp.quicksum(w[j] * x[j, b] for j in R(n)) <= c * y[b]
                  for b in R(k)), name="capacity")
    return m, x, y
```

The instance

On the instance of four items of weight $w = (5, 4, 3, 3)$ and capacity $c = 7$:

Bin packing on the instance

$$
\begin{aligned}
\min\quad & y_1 + y_2 + y_3 \\
\text{subject to}\quad & x_{11}+x_{12}+x_{13} = 1 \\
& x_{21} + x_{22} + x_{23} = 1 \\
& x_{31} + x_{32} + x_{33} = 1 \\
& x_{41} + x_{42} + x_{43} = 1 \\
& 5x_{11} + 4x_{21} + 3x_{31} + 3x_{41} - 7y_1 \le 0 \\
& 5x_{12} + 4x_{22} + 3x_{32} + 3x_{42} - 7y_2 \le 0 \\
& 5x_{13} + 4x_{23} + 3x_{33} + 3x_{43} - 7y_3 \le 0 \\
& x_{11},\ x_{12},\ x_{13},\ x_{21},\ x_{22},\ x_{23},\ x_{31},\ x_{32},\ x_{33},\ x_{41},\ x_{42},\ x_{43} \in \{0,1\} \\
& y_1,\ y_2,\ y_3 \in \{0,1\}
\end{aligned}
$$

The total weight is 15, so no solution can use fewer than $\lceil 15/7 \rceil = 3$ containers; the optimum uses exactly 3, and the count is therefore tight. In the model above the possible containers are three, the number actually needed: with six containers the model would have 42 columns and the last three would stay empty in every optimal solution.

> Relaxing y_b to $y_b \ge 0$ the model buys fractions of a container, and the LP optimum drops to $\sum_j w_j / c = 15/7 \approx 2.14$: the relaxation does not know that a container opens whole. This is why on this problem the dual bound is of little use and the heuristics matter more.

Makespan: the load of the busiest machine

Script: `python/cap06_cmax.py`

Makespan on identical machines

There are $n \in \mathbb{Z}_{\ge 1}$ jobs, of duration $d_j \in \mathbb{Q}_{>0}$, and $k \in \mathbb{Z}_{\ge 1}$ identical machines. Every job goes on a single machine and is not interrupted. Minimise the load of the busiest machine.

The model

With $x_{jm} = 1$ if job j goes on machine m, and $z \geq 0$ the load of the busiest machine:

Makespan on identical machines

$$\min \quad z \tag{3.2a}$$

$$\text{subject to} \quad \sum_{m=1}^{k} x_{jm} = 1, \qquad \forall j \in \{1, 2, \ldots, n\}, \tag{3.2b}$$

$$\sum_{j=1}^{n} d_j \, x_{jm} - z \leq 0, \qquad \forall m \in \{1, 2, \ldots, k\}, \tag{3.2c}$$

$$x_{jm} \in \{0, 1\}, \qquad \forall j \in \{1, 2, \ldots, n\}, \ \forall m \in \{1, 2, \ldots, k\}, \tag{3.2d}$$

$$z \geq 0. \tag{3.2e}$$

Description. The objective (3.2a) is the single variable z: no datum appears in it. It is the k load rows (3.2c) that give it meaning, saying that no machine works longer than z; the minimisation then presses it down onto the load of the busiest machine. It is the *min-max* technique of chapter 6. The constraints (3.2b) assign every job to a single machine, and the domains (3.2d) and (3.2e) declare the assignments binary and the load non-negative.

Building it in gurobipy

```
def modello_cmax(d, k):
    n = len(d)
    m = nuovo_modello("makespan")
    x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
    z = m.addVar(name="z")
    m.setObjective(z, GRB.MINIMIZE)
    m.addConstrs((x.sum(j, "*") == 1 for j in R(n)), name="job")
    m.addConstrs((gp.quicksum(d[j] * x[j, mm] for j in R(n)) <= z
                  for mm in R(k)), name="load")
    return m, x, z
```

The instance

On the instance of four jobs of duration $d = (3, 4, 5, 6)$ on $k = 2$ machines — the very one the heuristics chapter runs LPT on:

Makespan on the instance

$$
\begin{aligned}
\min \quad & & & & & & & & & & z & \\
\text{subject to} \quad & x_{11} & +x_{12} & & & & & & & & & = 1 \\
& & & x_{21} & +x_{22} & & & & & & & = 1 \\
& & & & & x_{31} & +x_{32} & & & & & = 1 \\
& & & & & & & x_{41} & +x_{42} & & & = 1 \\
& 3x_{11} & & +4x_{21} & & +5x_{31} & & +6x_{41} & & -z & \le 0 \\
& & 3x_{12} & & +4x_{22} & & +5x_{32} & & +6x_{42} & -z & \le 0 \\
& x_{11}, & x_{12}, & x_{21}, & x_{22}, & x_{31}, & x_{32}, & x_{41}, & x_{42} & & & \in \{0,1\} \\
& & & & & & & & & z & & \ge 0
\end{aligned}
$$

The total load is 18 and the machines are two: no solution can go below $18/2 = 9$, and the optimum is exactly 9 — the jobs split into $6+3$ and $5+4$. Here the count settles the problem on its own.

Travelling salesman: the MTZ formulation

Script: `python/cap06_tsp.py`

Travelling salesman

There are $n \in \mathbb{Z}_{\ge 2}$ cities and a distance $d_{ij} \in \mathbb{Q}_{>0}$ between every pair. Find the shortest tour that visits every city exactly once and returns to the starting point.

The model

With $x_{ij} = 1$ if the tour goes from i to j, the two families "one leaves once" and "one enters once" are not enough: they also admit solutions made of separate *subtours*. The Miller–Tucker–Zemlin formulation adds a variable u_i for every city other than the first, recording its position along the tour.

TSP, MTZ formulation

$$\min \quad \sum_{i=1}^{n} \sum_{j \neq i} d_{ij}\, x_{ij} \tag{3.3a}$$

$$\text{subject to} \quad \sum_{j \neq i} x_{ij} = 1, \qquad \forall i \in \{1, 2, \dots, n\}, \tag{3.3b}$$

$$\sum_{i \neq j} x_{ij} = 1, \qquad \forall j \in \{1, 2, \dots, n\}, \tag{3.3c}$$

$$u_i - u_j + n\, x_{ij} \le n - 1, \qquad \forall i, j \in \{2, 3, \dots, n\},\ i \neq j, \tag{3.3d}$$

$$x_{ij} \in \{0, 1\}, \qquad \forall i, j \in \{1, 2, \dots, n\},\ i \neq j, \tag{3.3e}$$

$$u_i \in [1,\, n-1], \qquad \forall i \in \{2, 3, \dots, n\}. \tag{3.3f}$$

Description. The objective (3.3a) sums the distances of the arcs travelled. The constraints (3.3b) and (3.3c) say that one leaves every city once and enters every city once. The constraints (3.3d) are the heart of the formulation. If $x_{ij} = 0$ the row becomes $u_i - u_j \leq n - 1$, always true because the u lie between 1 and $n - 1$: it forbids nothing. If instead $x_{ij} = 1$ it becomes $u_j \geq u_i + 1$, that is "if I go from i to j, the position of j is the next one". A subtour missing city 1 would need a chain of ever-growing positions closing on itself, which is impossible; city 1 has no u of its own precisely because it is where the tour closes.

Building it in gurobipy

```
def modello_tsp(D):
    n = len(D)
    m = nuovo_modello("tsp")
    x = m.addVars(((i, j) for i in R(n) for j in R(n) if i != j),
                  vtype=GRB.BINARY, name="x")
    u = m.addVars(R(1, n), lb=1, ub=n - 1, name="u")
    m.setObjective(gp.quicksum(D[i][j] * x[i, j] for i, j in x), GRB.
        MINIMIZE)
    m.addConstrs((gp.quicksum(x[i, j] for j in R(n) if j != i) == 1
                  for i in R(n)), name="out")
    m.addConstrs((gp.quicksum(x[i, j] for i in R(n) if i != j) == 1
                  for j in R(n)), name="in")
    m.addConstrs((u[i] - u[j] + n * x[i, j] <= n - 1
                  for i in R(1, n) for j in R(1, n) if i != j), name="
                     mtz")
    return m, x, u
```

The instance

On the four-city instance of chapter 4 the optimal tour is $1 \to 2 \to 4 \to 3 \to 1$ and measures 22. The model of the instance has fifteen columns — twelve arcs and three positions — and fourteen rows:

TSP on the instance

$$
\begin{aligned}
\min\ & 4x_{12}+5x_{13}+9x_{14}+4x_{21}+9x_{23}+9x_{24}+5x_{31}+9x_{32}+4x_{34}+9x_{41}+9x_{42}+4x_{43} \\
\text{subject to}\ & x_{12}+x_{13}+x_{14} = 1 \\
& x_{21}+x_{23}+x_{24} = 1 \\
& x_{31}+x_{32}+x_{34} = 1 \\
& x_{41}+x_{42}+x_{43} = 1 \\
& x_{21}+x_{31}+x_{41} = 1 \\
& x_{12}+x_{32}+x_{42} = 1 \\
& x_{13}+x_{23}+x_{43} = 1 \\
& x_{14}+x_{24}+x_{34} = 1 \\
& 4x_{23}+u_2-u_3 \le 3 \\
& 4x_{24}+u_2-u_4 \le 3 \\
& 4x_{32}-u_2+u_3 \le 3 \\
& 4x_{34}+u_3-u_4 \le 3 \\
& 4x_{42}-u_2+u_4 \le 3 \\
& 4x_{43}-u_3+u_4 \le 3 \\
& x_{12},\ x_{13},\ x_{14},\ x_{21},\ x_{23},\ x_{24},\ x_{31},\ x_{32},\ x_{34},\ x_{41},\ x_{42},\ x_{43} \in \{0,1\} \\
& u_2,\ u_3,\ u_4 \ge 1
\end{aligned}
$$

The MTZ constraints are $O(n^2)$ and take three lines of `gurobipy`, but their linear relaxation is weak: the continuous u absorb almost everything and the LP stays far from the integer optimum. Formulations that remove subtours with connectivity cuts give much better bounds, at the price of an exponential number of constraints to be generated as they are needed. For the sizes of this course MTZ is enough.

3.8 The course protocol, from start to finish

The sequence of every exercise

data → model → heuristic and check → LP and dual
→ MIP → table → figures and notebook

```
m, x = model(t, c, a)                                   # (1) data, (2) model

e = best_fit(t, a, lambda j, h, ra: c[j][h], "cost")    # (3) heuristic
ub = sum(c[j][h] for (j, h) in e.x)
assert ammissibile(m, {f"x[{j},{h}]": 1 for (j, h) in e.x})   # constraints, bounds AND integrality

d = dual(t, c, a)                                       # (4) the dual
```

```
    written by hand
by_hand = {f"mu[{j}]": min(c[j]) for j in range(n)}
lb, viol = valuta(d, by_hand);  assert viol <= 1e-9
zlp, zlp_str, pi = due_rilassamenti(m, d)                    #      checks
    strong duality

z = risolvi(m)                                                # (5) the MIP
row = registra_bound("assignment", ub, lb, zlp, zlp_str, z)        # (6)
    the table
salva_dati(pd.DataFrame([row]), "fam07_1_bound")             #      -> data
    /fam07_1_bound.csv
```

On the assignment instance of this chapter the protocol produces

$$LB = 10, \qquad z(LP) = z(LP^{+}) = \tfrac{53}{5}, \qquad z(MILP) = 11, \qquad UB = 11,$$

and the row ends up in `data/sched1_bound.csv`. That is where the notes, the website page and `check_numbers.py` read it from: **one single place where the number exists**.

The four numbers every exercise produces

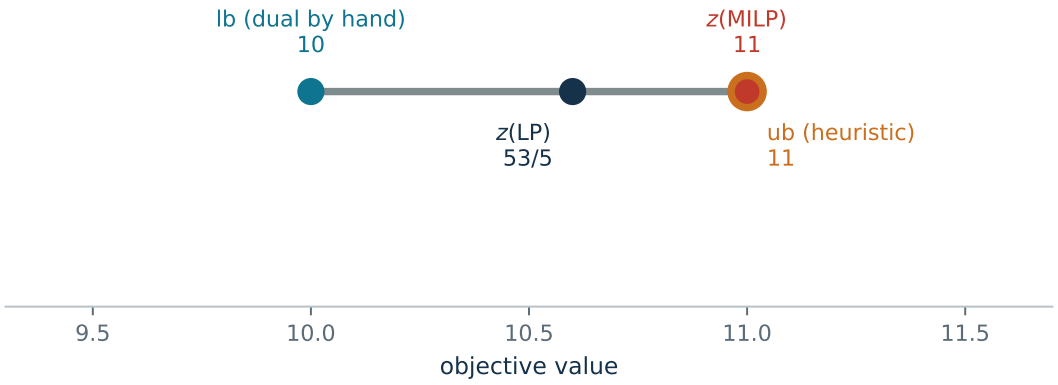


Figure 3.1: The four numbers of the protocol, on the assignment instance.

3.9 Running the code

Locally

From the `python/` folder:

```
python3 -m venv .venv && source .venv/bin/activate
pip install gurobipy pandas matplotlib
python3 fam07_1_assignment.py       # one chapter
python3 run_all.py                  # all of them, plus the notebooks
python3 check_numbers.py            # the asserts on every quoted number
```

On Colab

Every chapter has a notebook that opens from the badge on the website page. The first cell installs `gurobipy` and downloads the three shared modules (`stile.py`, `mip.py`, `euristiche.py`) if they are not already there: nothing needs to be installed on the computer.

The *size-limited* licence of the pip package allows models of up to **2000 variables and 2000 constraints**. The instances of this course are tiny — the largest model of the course has a few dozen variables — and all fit with an enormous margin. For larger instances the free academic licence is activated from portal.gurobi.com. If a model exceeds the limit, Gurobi reports it with an explicit error at `optimize()`: it does not silently produce a wrong result.

Chapter 4

Constructive heuristics

Script: `python/cap05_heuristics.py`, `python/euristiche.py` Online page

A constructive heuristic builds *one* solution, quickly, adding one element at a time and never backtracking. It proves nothing about the quality of that solution, and it is not even guaranteed to reach a feasible one: it can get stuck part-way, with an element that fits nowhere — precisely because it never backtracks. When it does end with a feasible solution, that solution is the other half of the sandwich of Chapter 2: the pessimistic side, the one guaranteed by a solution that really exists; when it fails, there is no primal bound.

What a heuristic must produce in this course

1. a readable **pseudocode**, with the scanning order, the choice criterion, the tie-breaking rule and the failure case stated explicitly;
2. the corresponding **Python function**, line by line (in `python/euristiche.py`);
3. the **trace** of the execution on an instance, which is the text appearing in the notes;
4. the **feasibility check** of the solution produced: constraints, bounds *and* integrality;
5. the resulting **bound**, with the right name: *UB* in a minimisation, *LB* in a maximisation.

Point 4 is not a formality. A solution satisfying the linear constraints but with a fractional component is feasible for the *relaxation*, not for the MILP, and its value is not a primal bound. The function `ammissibile` of `python/mip.py` checks integrality too, precisely for this reason.

The three ingredients

Every constructive heuristic is described by saying three things, and only three:

1. an **order** in which the elements to be placed are looked at;
2. an **insertion rule**, which says where the current element goes;
3. a **feasibility check**, which says when an insertion cannot be made.

Changing just one of the three gives a different heuristic, with a different value. That is why, when a heuristic is presented, the three ingredients have to be written out explicitly.

A heuristic produces a feasible solution, and its value is attainable. In a **minimisation** that value is an *upper* bound: $z(MILP) \le UB$. In a **maximisation** it is a *lower* bound: $LB \le z(MILP)$. Calling UB the result of a constructive heuristic on a maximisation is the commonest sign error in the course.

4.1 Bin packing: the insertion rules

The classical problem is **bin packing**, whose model is in Section 3.1: items must be put into identical bins of limited capacity, using as few of them as possible. Here it is not solved: it is *built*, one choice at a time.

Next-fit, first-fit, best-fit

Input: n, weights $w[1..n]$, capacity C, criterion γ
Output: the bins used

Function *Build*(n, w, C, γ)
  no bin open
  for $j := 1$ **to** n **do**
    `// next-fit:  the current bin only, then a new one is opened`
    `// first-fit:  the first open bin the item fits into`
    `// best-fit:  among the bins it fits into, the smallest` $\gamma(j, b, res)$
    choose b^* by the rule
    if *no open bin will do* **then** open a new one
    put j into b^*; $res[b^*] \leftarrow res[b^*] - w[j]$
  end
  return the bins used

All three scan the items *in the given order*: changing the order changes the result, and this must be said when a value is reported. Ties in the criterion γ are broken on the smallest index, so the run is reproducible.

The instance. Six items of weight $w = (4, 4, 5, 3, 2, 3)$, in bins of capacity $C = 7$: the same bins as the model of Section 3.1, with two more items. The instance there is for writing the model and is deliberately small — but for that very reason all three rules answer "three bins" on it, and cannot be told apart.

Heuristic	how it fills	UB	$z(MILP)$	gap
next-fit	`[4]` `[4]` `[5]` `[3+2]` `[3]`	5	3	66.7%
first-fit	`[4+3]` `[4+2]` `[5]` `[3]`	4	3	33.3%
best-fit on the fill	`[4+3]` `[4+3]` `[5+2]`	3	3	0.0%

The **elementary bound** settles it without the solver: the total weight is $4+4+5+3+2+3 = 21$ and one bin carries 7, so at least $\lceil 21/7 \rceil = 3$ bins are needed. Best-fit uses three, and fills all three exactly: $LB = UB = 3$, and the optimum is *proved* without ever calling the solver.

Where the three rules part company

Next-fit closes a bin as soon as an item does not fit, and never looks back: the last item, of weight 3, would fit exactly into the first bin, which still has 3 of room, but that bin is no longer looked at. Five bins instead of three.
Between **first-fit** and **best-fit** the difference is entirely about the item of weight 2. First-fit puts it in the *first* bin that takes it, the second, which is left with 1 of room now useless; best-fit puts it where it fits *exactly*, the third, which had 2. So the final 3 still finds room in the second, and the bins stay three.

4.2 $P||C_{\max}$: the least loaded rule

The second classic is **scheduling on identical machines**, also seen in Section 3.1, written $P||C_{\max}$ in standard notation: n jobs of duration t_j over k identical machines, minimising the instant the last one finishes. The natural rule is **list scheduling**: the current job goes to the machine that is least loaded at that moment, and the order in which the jobs are looked at decides the result. The order that works best is by **decreasing duration**, and the rule it gives has a name of its own, **LPT** (*longest processing time first*).

The idea

To minimise the *makespan* — the load of the busiest machine — over k identical machines: sort the jobs by **decreasing** duration and send each to the machine with the smallest current load. The decreasing order is essential: leaving the long jobs for last makes them impossible to place.

LPT — longest processing time

Input: n, k, durations $t[1..n]$
Output: an assignment x and its makespan

Function *LPT*(n, k, t)
  $L[m] \leftarrow 0$ for every m `// current loads`
  for j *in order of* decreasing $t[j]$ **do**
    $m^* \leftarrow \arg\min_m L[m]$ `// ties: the smallest index`
    $x[j][m^*] \leftarrow 1$; $L[m^*] \leftarrow L[m^*] + t[j]$
  end
  return x, $\max_m L[m]$

Seven jobs on three machines

$t = (5, 5, 4, 4, 3, 3, 3)$, $k = 3$, total 27. Trace:

- **Steps 1–3.** The jobs 5, 5, 4 go to the three empty machines: $L = (5, 5, 4)$.
- **Step 4.** Job 4: the smallest load is machine 3, which goes to 8. $L = (5, 5, 8)$.
- **Steps 5–6.** The two jobs of length 3 go to machines 1 and 2: $L = (8, 8, 8)$.
- **Step 7.** The last job of length 3 finds all loads equal to 8; by the tie rule it goes to

machine 1, which reaches 11.

LPT makespan: $UB = 11$, with loads $(11, 8, 8)$.
The elementary bound. The makespan is at least $\max\left(\max_j t_j,\ \sum_j t_j/k\right) = \max(5,\ 27/3) = 9$: no machine can do less than the longest job, and at least one must carry the average. The optimum is exactly $z(MILP) = 9$ — attained with $\{5, 4\}$, $\{5, 4\}$, $\{3, 3, 3\}$ — and LPT is off by 22.2%.

Two free bounds, to be compared

$\max_j t_j$ and $\sum_j t_j/k$ are two lower bounds computable without solving anything, and the better of the two is often already close to the optimum. An "obvious" bound that nobody writes down is a wasted bound: the dual of Chapter 2 is for when the obvious ones are not enough, not instead of them.

4.3 Set covering: the cheapest completion rule

Constructive covering heuristic

Input: costs $c[1..n]$, sets $S[1..m]$ (who covers zone i)
Output: a set of elements covering all zones

```
1 Function CoveringHeuristic(c, S)
2     uncovered ← {1, 2, ..., m};   y[j] ← 0 for every j
3     while uncovered ≠ ∅ do
4         for every j not yet chosen: new(j) ← |{i ∈ uncovered : j ∈ S_i}|
5         if new(j) = 0 for every j then return "no solution found"
6         j* ← arg min_{j : new(j)>0} c[j]/new(j)
7         y[j*] ← 1;   uncovered ← uncovered \ {i : j* ∈ S_i}
8     end
9     return y
```

The criterion is the **cost per newly covered zone**, not the absolute cost: picking the cheapest element regardless of how much it covers leads to far worse solutions.

The four teams of Chapter 2

Costs $c = (4, 3, 5, 3)$, six zones (the six pairs of districts).

- **Step 1.** All six zones uncovered; ratios 4/3, 3/3 = 1, 5/3, 3/3 = 1; a tie between elements 2 and 4, element 2 is chosen and covers zones 1, 2 and 5.
- **Step 2.** Zones 3, 4, 6 remain; ratios 4/2 = 2, 5/2, 3/2; element 4 is chosen and covers zones 4 and 6.
- **Step 3.** Zone 3 remains; ratios 4/1 and 5/1; element 1 is chosen.

Solution $\{1, 2, 4\}$, of cost $UB = 10$, which here is the optimum: $z(MILP) = 10$. The dual bound built in Chapter 2 is 7: the certified gap is 30%, even though the heuristic was

already optimal.

4.4 Knapsack: the best ratio rule

The **knapsack** is the model Chapter 3 opens with: the items have a *value* as well as a weight, and there is a single resource. The natural constructive rule looks at the ratio between the two, and what it produces is a feasible solution, hence a primal bound.

Constructive heuristic by value/weight ratio

Input: values $p[1..n]$, weights $w[1..n]$, capacity C
Output: a feasible subset

Function *KnapsackHeuristic*(p, w, C)
  $residual \leftarrow C$; $y[j] \leftarrow 0$ for every j
  for j *in order of* decreasing $p[j]/w[j]$ **do**
    if $w[j] \le residual$ **then** $y[j] \leftarrow 1$; $residual \leftarrow residual - w[j]$
  end
  return y

On the instance $p = (10, 7, 6, 4)$, $w = (5, 4, 3, 3)$, $C = 9$: the ratios are 2, 7/4, 2, 4/3; item 1 and then item 3 are taken (weight 8), items 2 and 4 no longer fit. Value 16. Since the problem is a *maximisation*,

$$LB = 16 \;\le\; z(MILP) = 17,$$

with a gap of 5.9%: the optimum takes items 1 and 2, filling the knapsack exactly. The ratio constructive heuristic goes wrong precisely because item 3, better by ratio, leaves an unusable residual.

4.5 TSP: the nearest neighbour

The fourth classic is the **travelling salesman problem** (TSP), whose variables Chapter 3 has given: given n cities and the distances $d_{ij} \in \mathbb{Q}_{>0}$ between every pair, find the shortest tour that visits them all once and returns to the start. It is the problem where step-by-step construction shows best, because the solution *is* a sequence: the order is the solution.

The classical constructive rule is the **nearest neighbour**: start from a city and each time go to the nearest among those not yet visited; when none are left, return to the start. It is feasible by construction — every city is entered once, and the last arc closes the tour — and it is fast, because at each step it looks only at the distances from the current city.

Five cities, five starting points

The distances, symmetric:

	1	2	3	4	5
1	—	5	2	2	9
2	5	—	4	3	4
3	2	4	—	4	7
4	2	3	4	—	7
5	9	4	7	7	—

Starting from city 1: the nearest is 3 (distance 2); from there 2 (4); then 4 (3); 5 is left (7); and the return to 1 costs 9. The tour $1 \to 3 \to 2 \to 4 \to 5 \to 1$ has length 25.
The last arc is the one that is paid for: the rule chooses well while it has a choice, and at the last step it has none. Changing the starting city changes the tour, and changes the length:

start	tour	length
1	$1 \to 3 \to 2 \to 4 \to 5 \to 1$	25
2	$2 \to 4 \to 1 \to 3 \to 5 \to 2$	18
3	$3 \to 1 \to 4 \to 2 \to 5 \to 3$	18
4	$4 \to 1 \to 3 \to 2 \to 5 \to 4$	19
5	$5 \to 2 \to 4 \to 1 \to 3 \to 5$	18

With five cities the distinct tours are $(5-1)!/2 = 12$, and they can all be enumerated: the optimum is $1 \to 3 \to 5 \to 2 \to 4 \to 1$, of length 18. Three starting points out of five find it, one stops at 19 and the one we started from at 25, that is 38.9% above the optimum.
Two things to take away. First: the heuristic gives *one* feasible solution, hence an upper bound — here $z(MILP) \leq 25$ — and nothing else; that 18 is the optimum is known by enumeration, not because the heuristic says so. Second: running the same rule from every start and keeping the best tour costs n times as much and gives a better upper bound. It is the simplest form of *multi-start*, and it does not change the nature of the thing: it is still a bound from one side only.

4.6 Lot sizing: least unit cost period covering

Least unit cost

Input: demands $d[1..T]$, setup cost f, unit holding cost h
Output: the periods in which production is launched, and the quantities

Function *LeastUnitCost*(d, f, h)
  $t \leftarrow 1$
  while $t \le T$ **do**
    skip the periods with $d[t] = 0$
    for $k := 1$ **to** $T - t + 1$ **do**
      $q_k \leftarrow \sum_{s=t}^{t+k-1} d[s]$; $c_k \leftarrow \left(f + h \sum_{s=t}^{t+k-1} (s-t)\, d[s]\right)/q_k$
    end
    $k^* \leftarrow \arg\min_k c_k$ `// the lowest average cost per unit`
    produce Q_{k^*} in period t; $t \leftarrow t + k^*$
  end

Wagner–Whitin is an **exact** dynamic-programming algorithm for the *uncapacitated* lot-sizing model: it solves that model to optimality in polynomial time. The procedure above is a heuristic, and its value is only a bound. Calling it "the Wagner–Whitin constructive heuristic" confuses two different things.

Five periods

Demands $d = (20, 10, 30, 40, 10)$, setup $f = 50$, holding $h = 1$.

- **Step 1.** From period 1 it pays to cover 2 periods (quantity 30, unit cost 2).
- **Step 2.** From period 3 it pays to cover 2 periods (quantity 70, unit cost ≈ 1.286).
- **Step 3.** From period 5 only that period remains (quantity 10, unit cost 5).

Cost of the heuristic solution: $UB = 200$. The optimum of the corresponding MILP is $z(MILP) = 170$, a gap of 17.6%: it is also the value Wagner–Whitin would give, being exact on this model.

4.7 Local search, and what it does not give

One step beyond construction

A **local search** starts from a feasible solution and tries elementary moves (move a job, swap two items), accepting those that improve. It stops at a **local optimum**: a solution that no move of the chosen family improves.

On the LPT solution of the example ($L = (11, 8, 8)$, makespan 11), the move "move one job to another machine" improves nothing: moving one of the two jobs of length 3 off machine 1 brings its load to 8 but raises the receiving machine to 11. The local search stops at 11, while the optimum is 9: to get there a *swap* between two machines is needed, not a simple move.

Local search returns a feasible solution, hence a bound on the pessimistic side, and nothing else. It proves nothing about the optimum, produces no bound on the optimistic side, and the fact that it stopped does not mean it has arrived. The only way to know how good it is, is to compare it with a bound from the other side.

4.8 When the constructive heuristic fails

A constructive heuristic can reach a point where the current element cannot be placed anywhere. It is not a programming mistake: it is the consequence of never backtracking.

Three jobs of duration $(3, 3, 2)$ on two machines with availability $(5, 3)$. Next-fit: job 1 goes to machine 1 (residual 2); job 2 does not fit and moves to machine 2 (residual 0); job 3 does not fit and there are no more machines: **failure**. But the problem is feasible: jobs 2 and 3 fit together on machine 1 ($3 + 2 = 5$) and job 1 on machine 2 ($3 \leq 3$).
A constructive heuristic is *myopic*: it decides one thing at a time and never backtracks. Its failure is information about the heuristic, not about the problem. To prove that a model is infeasible one needs the solver (`Status = INFEASIBLE`) or a proof, not a constructive heuristic that went wrong.

If the heuristic returns no solution, nothing has been obtained: there is no primal bound. The two things to do are, in this order:

1. change the order — the order by decreasing size fails far less often than the natural one, because it places the difficult elements first;
2. change the insertion rule.

If nothing is found even so, the question becomes whether the problem is feasible, and that is a question for the model, not for the heuristic.

4.9 What it gives, and what it does not

The exact account of what is known

Let $\bar{z}$ be the value of the solution found by a constructive heuristic.

- In a **minimisation** problem one knows that
$$z(MILP) \leq \bar{z},$$
that is, $\bar{z}$ is an upper bound: $UB = \bar{z}$.
- In a **maximisation** problem one knows that
$$z(MILP) \geq \bar{z},$$
that is, $\bar{z}$ is a lower bound: $LB = \bar{z}$.

- In neither case does one know *how far* $\bar{z}$ is from the optimum. To know that, a bound from the other side is needed, and that one does not come from a feasible solution: it comes from a relaxation or from a combinatorial argument (Chapter 2).

With both bounds in hand one can write the **certified gap**: in absolute terms $UB - LB$, and as a percentage

$$\frac{UB - LB}{|UB|}$$

in a minimisation, where UB is the heuristic solution, and $(UB - LB)/|LB|$ in a maximisation, where it is LB that plays the incumbent. The denominator is always the solution one actually holds, never $z(MILP)$: that is what makes it the only measure of quality available *before* solving the MILP.

Why build the heuristic by hand, if the solver is there

A modern solver finds feasible solutions on its own, often better ones. Building a heuristic by hand is still worth it, for three reasons:

- it forces one to understand what makes a solution feasible, and that is exactly the content of the constraints;
- it gives a reference value independent of the solver: if the solver returns something worse, there is a mistake in the model;
- on large problems the solver does not reach the optimum, and the quality of the final solution depends on the initial one.

4.10 The overview of the heuristics

Heuristic	Direction	value	$z(MILP)$	gap
next-fit (bin packing)	min (UB)	5	3	66.7%
first-fit (bin packing)	min (UB)	4	3	33.3%
best-fit on the fill (bin packing)	min (UB)	3	3	0.0%
LPT (makespan)	min (UB)	11	9	22.2%
covering constructive heuristic	min (UB)	10	10	0.0%
ratio constructive heuristic (knapsack)	max (LB)	16	17	5.9%
nearest neighbour (TSP)	min (UB)	25	18	38.9%
least unit cost (lot sizing)	min (UB)	200	170	17.6%

What this table teaches

Two heuristics find the optimum and six do not, and *before* solving the MILP there is no way of knowing which. A 0% gap and a 67% gap are told apart only *afterwards*. This is exactly why the course always asks for two bounds: a heuristic on its own says how much a solution one can actually implement costs, not how much is being lost.

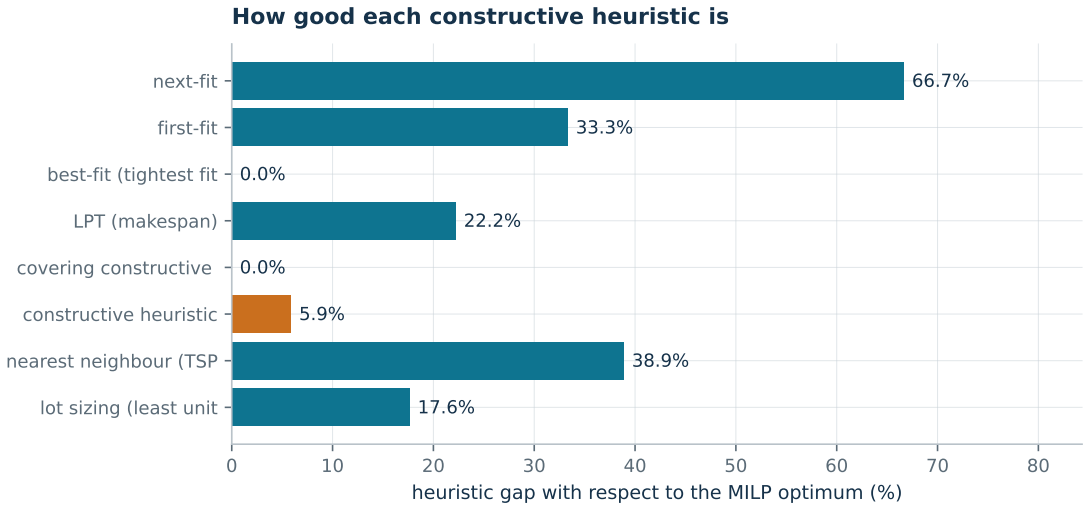


Figure 4.1: The gap of each heuristic with respect to the MILP optimum, on the minimal instances of this chapter. Teal for minimisation problems, orange for maximisation.

Chapter 5

Logic and binary variables

Script: `python/cap02_logic.py`, `python/booleane.py` Online page

A binary variable is a "yes/no" answer. This chapter shows how to translate logical conditions between those answers into *linear constraints* and — above all — how to *prove* that the translation is exact.

5.1 Propositions and binary variables

One proposition, one variable

A **boolean function** returns one of the two values `TRUE` and `FALSE`; it is represented by a binary variable

$$x \in \{0,1\}, \qquad x = 1 \iff \text{the proposition is } \texttt{TRUE}, \qquad x = 0 \iff \text{it is } \texttt{FALSE}.$$

A **boolean expression** is built from binary variables, the three operators `AND` (conjunction, $\wedge$), `OR` (disjunction, $\vee$) and `NOT` (negation, $\neg$), and parentheses.

Two boolean expressions

$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big),$$
$$(x_a\ \ \texttt{OR}\ \ x_b)\ \ \texttt{AND}\ \ (x_c\ \ \texttt{OR}\ \ x_d)\ \ \texttt{AND}\ \ \texttt{NOT}\ x_e.$$

Satisfiability

The **boolean satisfiability problem** asks whether there is an assignment of logical values to the variables that makes the expression true. If there is, the expression is **satisfiable**; otherwise it is **unsatisfiable**.

Satisfiable and not

- the formula
$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big)$$
is satisfied, for instance, by $x_a = 0$ (the others arbitrary).

- the formula
$$(x_a \texttt{ OR } x_b) \texttt{ AND } (x_c \texttt{ OR } x_d) \texttt{ AND NOT } x_e$$
is satisfied by $x_a = 1$, $x_c = 1$, $x_b = x_d = x_e = 0$.
- $(x_a \texttt{ OR } x_b) \texttt{ AND } (\texttt{NOT } x_a \texttt{ OR } x_b) \texttt{ AND } (x_a \texttt{ OR NOT } x_b) \texttt{ AND } (\texttt{NOT } x_a \texttt{ OR NOT } x_b)$ is unsatisfiable: each of the four possible assignments of (x_a, x_b) falsifies one of the four clauses.

5.2 Literals, clauses, conjunctive normal form

The three words

A **literal** is a variable (x_a, positive literal) or its negation ($\texttt{NOT } x_a$, negative literal). A **clause** is a disjunction (OR) of literals, or a single literal. An expression is in **conjunctive normal form** (CNF) if it is a conjunction (AND) of clauses, or a single clause.

Of the two expressions in the previous example, the second is in CNF and the first is not. Every boolean expression *can* be rewritten in CNF, using the equivalences below.

The equivalences that are needed

For every triple of binary variables $x_a, x_b, x_c \in \{0, 1\}$:

$$x_a \texttt{ AND } (x_b \texttt{ OR } x_c) \iff (x_a \texttt{ AND } x_b) \texttt{ OR } (x_a \texttt{ AND } x_c) \tag{5.1}$$
$$x_a \texttt{ OR } (x_b \texttt{ AND } x_c) \iff (x_a \texttt{ OR } x_b) \texttt{ AND } (x_a \texttt{ OR } x_c) \tag{5.2}$$
$$\texttt{NOT } (x_a \texttt{ OR } x_b) \iff \texttt{NOT } x_a \texttt{ AND NOT } x_b \tag{5.3}$$
$$\texttt{NOT } (x_a \texttt{ AND } x_b) \iff \texttt{NOT } x_a \texttt{ OR NOT } x_b \tag{5.4}$$
$$x_a \texttt{ OR } (x_a \texttt{ AND } x_b) \iff x_a \tag{5.5}$$
$$x_a \texttt{ AND } (x_a \texttt{ OR } x_b) \iff x_a \tag{5.6}$$
$$\texttt{NOT } (\texttt{NOT } x_a) \iff x_a \tag{5.7}$$

Equations (5.1)–(5.2) are the **distributive** properties, (5.3)–(5.4) are **De Morgan**'s laws, (5.5)–(5.6) are **absorption**. They hold with more than two terms inside the parentheses and with expressions in place of the variables.

Every equivalence is proved by cases: with two variables there are 4 assignments, with three there are 8, and one checks that the two sides agree on each. The script `cap02_logic.py` runs this check on all seven equivalences.

Collecting: (5.1) read backwards

Read from right to left, the distributive property (5.1) allows two conjunctions sharing a literal to be *collected*:

$$(x_a \texttt{ AND } x_b) \texttt{ OR } (x_a \texttt{ AND } x_c) \iff x_a \texttt{ AND } (x_b \texttt{ OR } x_c).$$

It is useful when putting into CNF a disjunction of several conjunctions with common literals: it reduces, step by step, the number of terms to distribute.

Two transformations into CNF

- for the formula

$$\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ \big((x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_d\ \ \texttt{OR}\ \ x_e)\big)$$

the distributive (5.2) is enough, and it becomes

$$(\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (\texttt{NOT}\ x_a\ \ \texttt{OR}\ \ x_d\ \ \texttt{OR}\ \ x_e).$$

- the formula

$$(x_a\ \ \texttt{AND}\ \ x_b)\ \ \texttt{OR}\ \ (x_a\ \ \texttt{AND}\ \ x_c)\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c)$$

says "at least two of a, b, c". Collect x_a in the first two,

$$\big[x_a\ \ \texttt{AND}\ \ (x_b\ \ \texttt{OR}\ \ x_c)\big]\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c),$$

apply (5.2) and obtain the two factors

$$x_a\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c)\qquad\text{and}\qquad(x_b\ \ \texttt{OR}\ \ x_c)\ \ \texttt{OR}\ \ (x_b\ \ \texttt{AND}\ \ x_c);$$

the second reduces to $(x_b\ \ \texttt{OR}\ \ x_c)$ by absorption (5.5), and the first is distributed once more. Altogether:

$$(x_a\ \ \texttt{OR}\ \ x_b)\ \ \texttt{AND}\ \ (x_a\ \ \texttt{OR}\ \ x_c)\ \ \texttt{AND}\ \ (x_b\ \ \texttt{OR}\ \ x_c).$$

This will be the key to exercise 2.4.

5.3 From CNF to linear constraints

The translation, in three rules

A boolean expression in CNF is equivalent to a system of linear constraints on the same binary variables:

1. every **clause** becomes an inequality constraint ≥ 1;
2. every $\texttt{OR}$ inside the clause becomes a $+$;
3. every negative literal $\texttt{NOT}\ x$ becomes $1 - x$.

Why it is exact

A clause with positive literals P and negative literals N becomes

$$\sum_{i\in P} x_i + \sum_{i\in N}(1 - x_i) \geq 1 \qquad\Longleftrightarrow\qquad \sum_{i\in P} x_i - \sum_{i\in N} x_i \geq 1 - |N|.$$

On a binary variable, $x_i = 1$ contributes 1 to the left-hand side if the literal is positive and satisfied, and $1 - x_i = 1$ contributes 1 if the literal is negative and satisfied. The sum counts *how many literals of the clause are true*: asking it to be ≥ 1 is exactly asking the clause to

be true. The conjunction of clauses becomes the system of all the constraints, because they must all hold together.

With two or more negative literals the equivalent form obtained by multiplying by -1 reads better: for instance $1 - x_1 + 1 - x_6 + x_7 \geq 1$ is written $x_1 + x_6 - x_7 \leq 1$. It is the same constraint. What never changes is the number of constraints: *one per clause*, and clauses are counted after simplification (removing the tautologies x `OR` `NOT` x and the clauses absorbed by a subclause of theirs).

Three translations

$$(\texttt{NOT}\ x_a\ \texttt{OR}\ x_b\ \texttt{OR}\ x_c)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ x_d\ \texttt{OR}\ x_e) \iff \begin{cases} -x_a + x_b + x_c \geq 0, \\ -x_a + x_d + x_e \geq 0; \end{cases}$$

$$(x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (x_c\ \texttt{OR}\ x_d)\ \texttt{AND}\ \texttt{NOT}\ x_e \iff \begin{cases} x_a + x_b \geq 1, \\ x_c + x_d \geq 1, \\ x_e \leq 0; \end{cases}$$

$$(x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ x_b)\ \texttt{AND}\ (x_a\ \texttt{OR}\ \texttt{NOT}\ x_b)\ \texttt{AND}\ (\texttt{NOT}\ x_a\ \texttt{OR}\ \texttt{NOT}\ x_b)$$
$$\iff \begin{cases} x_a + x_b \geq 1, \\ -x_a + x_b \geq 0, \\ x_a - x_b \geq 0, \\ x_a + x_b \leq 1. \end{cases}$$

The last system has no binary solution, just as the expression was unsatisfiable: the first three constraints force $x_a = x_b = 1$, which violates the fourth.

5.4 Logical implications

The implication between two variables

The logical implication $x_a \Rightarrow x_b$ ("if a then b") is equivalent to the boolean expression

$$\texttt{NOT}\ x_a\ \texttt{OR}\ x_b,$$

already in CNF, and hence to the linear constraint

$$1 - x_a + x_b \geq 1 \iff x_b - x_a \geq 0.$$

The contrapositive is the same thing

By contraposition, $x_a \Rightarrow x_b$ also implies the contrapositive

$$\texttt{NOT}\ x_b \Rightarrow \texttt{NOT}\ x_a.$$

This is not a second constraint: its boolean expression is

$$\texttt{NOT}\,(\texttt{NOT}\ x_b)\quad \texttt{OR}\quad \texttt{NOT}\ x_a,$$

which by double negation (5.7) is the same

$$\texttt{NOT}\ x_a\quad \texttt{OR}\quad x_b.$$

A single constraint imposes both readings — and when describing a model it is best to write the one that reads better in the application at hand.

$x_b - x_a \geq 0$ says "if a then b", not "a if and only if b": it admits $x_a = 0$, $x_b = 1$. An "if and only if" needs *two* implications, that is $x_a = x_b$, or one implication plus an optimality argument: that is the written when one of the two implications follows from the sign of the coefficient in the objective.

The same recipe works when antecedent and consequent are arbitrary boolean expressions: one writes NOT (antecedent) OR (consequent), puts it into CNF and translates.

The four basic combinations

Implication	Expression in CNF	Constraints	#
x_a AND $x_b \Rightarrow x_c$	NOT x_a OR NOT x_b OR x_c	$x_a + x_b - x_c \leq 1$	1
x_a OR $x_b \Rightarrow x_c$	(NOT x_a OR x_c) AND (NOT x_b OR x_c)	$x_c - x_a \geq 0$, $x_c - x_b \geq 0$	2
$x_a \Rightarrow x_b$ AND x_c	(NOT x_a OR x_b) AND (NOT x_a OR x_c)	$x_b - x_a \geq 0$, $x_c - x_a \geq 0$	2
$x_a \Rightarrow x_b$ OR x_c	NOT x_a OR x_b OR x_c	$x_b + x_c - x_a \geq 0$	1

Read the table like this: a *disjunction in the antecedent* and a *conjunction in the consequent* cost two constraints; a conjunction in the antecedent and a disjunction in the consequent cost one. This is why the aggregated constraint $\sum_j x_{jm} \leq k\, y_m$ and the disaggregated one $x_{jm} \leq y_m$ have different costs in number of rows — and different strengths in the relaxation.

An antecedent with three literals

$$x_a\ \texttt{AND}\ \texttt{NOT}\ x_b\ \texttt{AND}\ x_c \Rightarrow \texttt{NOT}\ x_d \iff \texttt{NOT}\ x_a\ \texttt{OR}\ x_b\ \texttt{OR}\ \texttt{NOT}\ x_c\ \texttt{OR}\ \texttt{NOT}\ x_d$$
$$\iff\quad x_a - x_b + x_c + x_d \leq 2.$$

5.5 Splitting an implication: when it is allowed

Two valid splits and one that is not

$$(x_a \ \texttt{OR} \ x_b) \Rightarrow x_c \iff (x_a \Rightarrow x_c) \ \texttt{AND} \ (x_b \Rightarrow x_c) \tag{5.8}$$
$$x_a \Rightarrow (x_b \ \texttt{AND} \ x_c) \iff (x_a \Rightarrow x_b) \ \texttt{AND} \ (x_a \Rightarrow x_c) \tag{5.9}$$

The analogous split with a **conjunctive** antecedent is *not* valid:

$$(x_a \ \texttt{AND} \ x_b) \Rightarrow x_c \quad \textbf{not} \iff \quad (x_a \Rightarrow x_c) \ \texttt{AND} \ (x_b \Rightarrow x_c).$$

With $x_a = 1$, $x_b = 0$, $x_c = 0$ the implication $(x_a \ \texttt{AND} \ x_b) \Rightarrow x_c$ is *true* — its antecedent is `FALSE` — but $x_a \Rightarrow x_c$ is *false*. The conjunction of the two split implications is therefore **strictly stronger** than the original: it cuts off solutions the problem allows. Getting this wrong does not produce a "slightly different" model, it produces a model that may exclude the optimum.

The two valid splits are often the shortest route: in the previous table, the rows with two constraints follow directly from (5.8) or (5.9), without going through De Morgan and distributivity.

5.6 Counting: at most one, at least one, exactly one

Cardinality conditions on a set I of binary variables are already linear and need no CNF.

Condition	Constraint	Note
at least one	$\sum_{i\in I} x_i \geq 1$	it is the clause $\bigvee_{i\in I} x_i$: *set covering*
at most one	$\sum_{i\in I} x_i \leq 1$	*set packing*; equivalent to the $\binom{\lvert I\rvert}{2}$ clauses $\texttt{NOT}\ x_i \ \texttt{OR} \ \texttt{NOT}\ x_j$, but in a single constraint and with a tighter relaxation
exactly one	$\sum_{i\in I} x_i = 1$	*set partitioning*: the two above together
at least p	$\sum_{i\in I} x_i \geq p$	in CNF it would take $\binom{\lvert I\rvert}{\lvert I\rvert-p+1}$ clauses
at most p	$\sum_{i\in I} x_i \leq p$	in CNF it would take $\binom{\lvert I\rvert}{p+1}$ clauses
none	$\sum_{i\in I} x_i \leq 0$	that is, $x_i = 0$ for every i

One cardinality constraint beats many clauses

"At most one out of three" can be written as three clauses,

$$\texttt{NOT}\ x_1 \ \texttt{OR} \ \texttt{NOT}\ x_2, \qquad \texttt{NOT}\ x_1 \ \texttt{OR} \ \texttt{NOT}\ x_3, \qquad \texttt{NOT}\ x_2 \ \texttt{OR} \ \texttt{NOT}\ x_3,$$

that is $x_1 + x_2 \leq 1$, $x_1 + x_3 \leq 1$, $x_2 + x_3 \leq 1$; or as the single $x_1 + x_2 + x_3 \leq 1$. The two systems have the *same* binary solutions, but not the same relaxation: $x_1 = x_2 = x_3 = 1/2$ satisfies the three clauses and violates the aggregated constraint. The compact version is shorter *and* stronger: a case with no trade-off. When counting is possible, count; CNF is for conditions that are not cardinalities.

5.7 Logical operators and set operators

Associating with every binary variable a subset of a universe U — $x_a = 1$ means "the chosen element lies in A" — the logical operators become set operators:

Logic	Sets	
x_a `AND` x_b	$A \cap B$	intersection
x_a `OR` x_b	$A \cup B$	union
`NOT` x_a	$\overline{A}$	complement

The equivalences of the previous section become the set identities $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$, $\overline{A \cap B} = \overline{A} \cup \overline{B}$, $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $A \cup (A \cap B) = A$ and $A \cap (A \cup B) = A$. Those who prefer Venn diagrams thus get a second proof of the same rules; those who prefer enumeration have the first. Each checks the other.

5.8 Checking the translation, not trusting it

The operational criterion

A translation into linear constraints is correct when, for *every* binary assignment of the variables involved, the expression is true if and only if all the constraints are satisfied. With few variables this check is an enumeration of 2^n cases, done in an instant: it is the proof, by cases, of the result. The module `python/booleane.py` performs it — and every translation in the following section is checked this way.

The module also does the rest of the chapter's work: it eliminates implications ($A \Rightarrow B$ becomes `NOT` A `OR` B), pushes negations onto the literals with De Morgan, distributes `OR` over `AND`, discards tautological and absorbed clauses, and finally applies the three translation rules. It is not a SAT solver: it is a line-by-line transcription of this chapter's rules.

5.9 Five exercises

In all five, the projects are numbered and for every project p the binary variable is

$$x_p = \begin{cases} 1 & \text{if project } p \text{ is chosen,} \\ 0 & \text{otherwise.} \end{cases}$$

The solutions — CNF expression, linear constraints, constraint count and the number of assignments on which the equivalence is checked by enumeration — are in the solutions document, reserved for teachers.

Exercise 5.1 — direct implications

Ten projects. Model as linear constraints:

1. if project 2 is chosen then project 3 is chosen;
2. if project 2 is chosen then project 4 is not chosen;
3. if projects 1 and 6 are chosen then project 7 is chosen;

4. if project 1 or project 6 is chosen then project 8 is chosen;
5. if projects 2 and 3 are chosen then project 9 is not chosen;
6. if project 2 or project 3 is chosen then project 10 is not chosen.

Exercise 5.2 — negated antecedents and consequents

Ten projects. Model:

1. if project 3 is not chosen then project 2 is chosen;
2. if project 4 is not chosen then project 2 is not chosen;
3. if project 7 is chosen then projects 1 and 6 are chosen;
4. if project 8 is chosen then project 1 or project 6 is chosen;
5. if project 9 is not chosen then projects 2 and 3 are chosen;
6. if project 10 is not chosen then project 2 or project 3 is chosen.

Exercise 5.3 — compound antecedents and consequents

Eight projects. Model:

1. x_7 OR $x_3 \Rightarrow x_1$ AND x_2;
2. x_1 AND x_6 AND $x_7 \Rightarrow x_8$;
3. x_5 AND x_2 AND NOT $x_4 \Rightarrow$ NOT x_3;
4. (x_1 OR x_4) AND $x_6 \Rightarrow x_2$ AND (x_5 OR x_7);
5. (x_2 OR x_5) AND NOT $x_8 \Rightarrow x_3$ OR NOT x_6;
6. (x_1 OR x_4) AND (x_2 OR x_5) AND NOT $x_8 \Rightarrow x_3$ AND (NOT x_6 OR x_7).

Exercise 5.4 — "at least two of"

Nine projects. Model:

1. if project 4 is chosen then at least two of 1, 2, 3;
2. if at least two of 6, 7, 8 then project 5;
3. if project 4 is not chosen then at least two of 1, 2, 3, 9;
4. if project 8 is chosen then (1 and 6) or (1 and 7) or (2 and 6);
5. if at least two of 1, 3, 5 then project 9 is not chosen;
6. if (1 and 2) or (3 and 4) then project 5.

The condition "at least two of 1, 2, 3" is also $x_1 + x_2 + x_3 \geq 2$. As the consequent of the implication governed by x_4 it becomes

$$x_1 + x_2 + x_3 \geq 2x_4 :$$

a single constraint, against the three clauses of item 1. The two systems have the same 16 binary solutions — checked by enumeration — but not the same relaxation, and this time it is the counted form that is **stronger**. Summing the three clauses $x_i + x_j \geq x_4$ gives only $x_1 + x_2 + x_3 \geq \frac{3}{2}x_4$, weaker than $\geq 2x_4$; conversely, with $0 \leq x_i \leq 1$, the counted form implies each clause. On max $x_1 + x_2 + x_3 + 3x_4$ subject to $x_1 + x_2 + x_3 + 2x_4 \leq 3$, where $z(MILP) = 3$, the three clauses give $z(LP^+) = 27/7 \approx 3.86$ and the counted constraint gives $z(LP^+) = 15/4 = 3.75$: closer to the integer optimum.
The moral is not "counting is always better": in the previous section "at most one" counted was stronger than the pairwise clauses, here "at least two" counted is again stronger, but it does not always go that way — an aggregated sum can perfectly well be *weaker* than the inequalities it summarises. The moral is that two formulations with the same integer set must be compared on their relaxation, by solving it, and not by eye.

Exercise 5.5 — splits

Ten projects. Model:

1. x_1 `OR` $x_2 \Rightarrow x_3$;
2. $x_4 \Rightarrow x_5$ `AND` x_6;
3. x_1 `OR` $x_2 \Rightarrow x_3$ `AND` x_4;
4. x_1 `AND` $x_2 \Rightarrow x_3$;
5. x_5 `OR` $x_6 \Rightarrow$ `NOT` x_7;
6. `NOT` x_8 `OR` `NOT` $x_9 \Rightarrow x_{10}$.

5.10 Logical constraints inside an optimisation model

The constraints of this chapter rarely live alone: they are added to a model that optimises something. With the $n = 10$ projects of exercise 5.9, revenues r_p and costs c_p

project p	1	2	3	4	5	6	7	8	9	10
revenue r_p	9	7	4	8	3	6	2	5	7	6
cost c_p	4	3	2	4	2	3	1	3	4	3

and budget $b = 14$, the model is

Project selection with logical constraints

$$\max \quad \sum_{p=1}^{n} r_p\, x_p$$

$$\text{subject to} \quad \sum_{p=1}^{n} c_p\, x_p \le b,$$

the 8 constraints of exercise 5.9,

$$x_p \in \{0,1\}, \qquad \forall p \in \{1,2,\dots,n\}.$$

Description. The objective adds up the revenues of the chosen projects. The first constraint, a single one, is the budget. The eight recalled constraints are the linear translations of the six implications of exercise 5.9: they add no variables, only rows. The domains are the n binary conditions, one per project.

The model on the instance. On the data above, one column per variable:

Project selection, ten binaries

$$\begin{aligned}
&\max\ 9x_2{+}7x_3{+}4x_4{+}8x_5{+}3x_6{+}6x_7{+}2x_8{+}5x_9{+}7x_{10}{+}6x_{11}\\
&\text{subject to } 4x_2{+}3x_3{+}2x_4{+}4x_5{+}2x_6{+}3x_7\ {+}x_8{+}3x_9{+}4x_{10}{+}3x_{11} \le 14\\
&\qquad -x_3\ {+}x_4 \ge 0\\
&\qquad x_3\ {+}x_5 \le 1\\
&\qquad x_2\ {+}x_7\ {-}x_8 \le 1\\
&\qquad -x_2\ {+}x_9 \ge 0\\
&\qquad -x_7\ {+}x_9 \ge 0\\
&\qquad x_3\ {+}x_4\ {+}x_{10} \le 2\\
&\qquad x_3\ {+}x_{11} \le 1\\
&\qquad x_4\ {+}x_{11} \le 1\\
&x_2,\ x_3,\ x_4,\ x_5,\ x_6,\ x_7,\ x_8,\ x_9,\ x_{10},\ x_{11} \in \{0,1\}
\end{aligned}$$

Without the logical constraints the optimum is 30. With them the optimum drops to $z(MILP) = 28$, obtained by choosing projects $1, 2, 3, 5, 8$, of total cost 14: the budget is tight. The relaxation $z(LP^+)$ is 29. The six implications hold: 2 is chosen and so is 3; 4 is not chosen; 1 and 6 are not both chosen; 1 is chosen and so is 8; 2 and 3 are chosen and 9 is not; 2 is chosen and 10 is not.

How much six implications cut

The $2^{10} = 1024$ assignments of the ten binaries drop to 234 once all six implications of exercise 5.9 are imposed: less than a quarter. Figure 5.1 shows how many survive each implication on its own and how many survive as they accumulate. None of these implications, on its own, cuts more than half the space; together they cut three quarters. This is why a model is written one constraint at a time but judged as a whole.

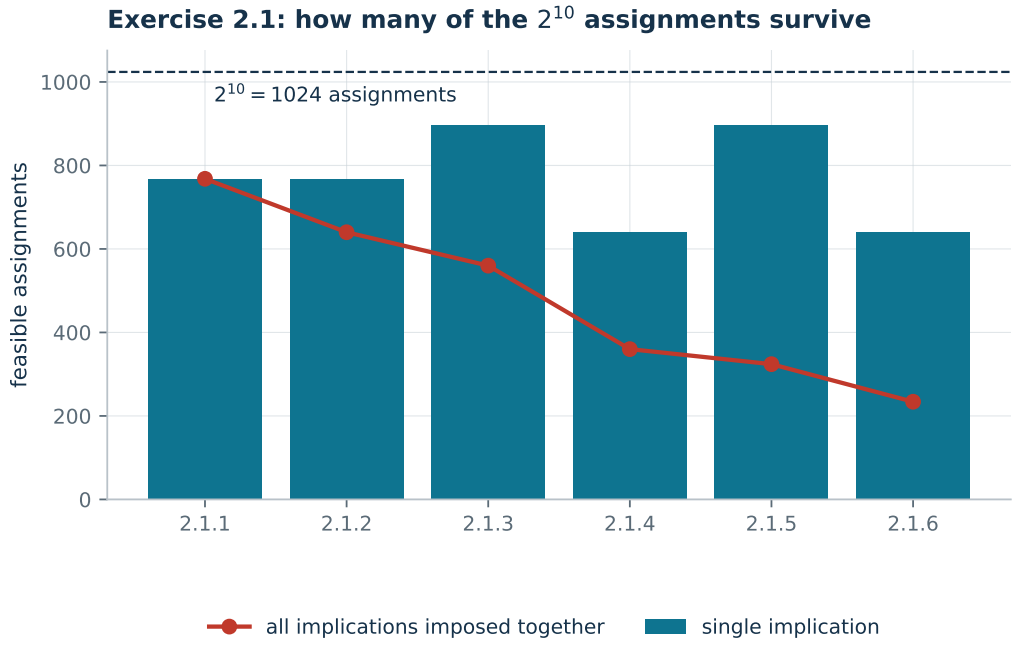


Figure 5.1: Exercise 5.9: assignments of the ten binaries surviving each implication on its own (bars) and all the implications imposed up to that point (line).

The model in gurobipy

In the code that follows, next to `gurobipy`, there is the small teaching module `booleane.py`, distributed with the course material: it only serves to write a logical formula the way one would write it by hand — `V` creates a variable, `AND`, `OR`, `NOT` and `IMP` compose it — and to convert it into conjunctive normal form with `cnf`. The function `vincolo` then turns a single clause into the coefficients, the direction and the right-hand side of the corresponding inequality. All it does is the procedure of this chapter, written once.

```
from booleane import cnf, vincolo, IMP, AND, OR, NOT, V

x = {p: V(f"x{p}") for p in range(1, 11)}
implications = [IMP(x[2], x[3]), IMP(x[2], NOT(x[4])),
                IMP(AND(x[1], x[6]), x[7]), IMP(OR(x[1], x[6]), x[8]),
                IMP(AND(x[2], x[3]), NOT(x[9])), IMP(OR(x[2], x[3]),
                   NOT(x[10]))]

m = gp.Model("project_selection");  m.Params.OutputFlag = 0
xv = m.addVars(range(1, 11), vtype=GRB.BINARY, name="x")
m.setObjective(gp.quicksum(r[p] * xv[p] for p in range(1, 11)), GRB.
    MAXIMIZE)
m.addConstr(gp.quicksum(b[p] * xv[p] for p in range(1, 11)) <= budget,
    name="budget")
for i, formula in enumerate(implications, 1):          # one clause,
    one constraint
    for j, clause in enumerate(cnf(formula), 1):
        coef, sense, rhs = vincolo(clause)
        lhs = gp.quicksum(k * xv[int(n[1:])] for n, k in coef.items())
        m.addConstr(lhs <= rhs if sense == "<=" else lhs >= rhs, name=f
```

```
            "logic{i}_{j}")
m.optimize()
```

The two bounds of this course — the primal one and the dual one — reproduce, in teaching form, the two sides of the sandwich that a MIP solver tries to close.

5.11 What this chapter has put in your hands

A logical condition between yes/no decisions is always written the same way: one binary per elementary fact, the condition brought into conjunctive normal form — a conjunction of ORs — and every clause becomes a linear inequality. The AND costs nothing, because constraints are already in AND with each other; the OR becomes a sum ≥ 1; the NOT becomes $1 - x$; the implication $x \Rightarrow y$ becomes $x \leq y$; the "if and only if" asks for both inequalities.

Three shapes recur often enough to have a name: the *set covering* ($\sum_{j \in S} x_j \geq 1$, at least one), the *set packing* (≤ 1, at most one) and the *set partitioning* ($= 1$, exactly one). Recognising them in a statement is half of the modelling work.

Chapter 6

Links between variables

Script: `python/cap03_links.py` Online page

Chapter 5 links variables that are *all binary*. Here we link *different* families: binary with continuous, binary with integer, continuous with each other. There are fourteen techniques, and they are the real content of MIP modelling: almost every model in this course is a combination of them.

How to read each section

Every technique is presented in five steps:

(a) the **link in words**: what one wants to say;

(b) the **linear constraints** that say it, with their count;

(c) the **proof**: in both directions if both are imposed by the constraints; with an exchange argument if one follows from optimality; with a *counterexample* when a converse is false;

(d) the **strength of the relaxation**, on a minimal instance solved by the script;

(e) the `gurobipy` **line** that writes it in the solver.

Step (c) is what tells a correct model from a plausible one. Step (d) is what tells a model that can be solved from one that cannot.

Throughout the chapter we distinguish:

- a property **imposed by the constraints**: it holds for *every* feasible solution, optimal or not. It is proved by looking at the constraints alone;
- a property **of optimality**: it holds only in optimal solutions. It is proved by an *exchange argument*: take a feasible solution that violates it, build a modified solution, check that it stays feasible and compare the values. If the value improves *strictly*, the conclusion is "in every optimum"; if it merely does not worsen, the conclusion is "there exists an optimum", which is weaker.

Writing "in every optimum" when the coefficient is only non-negative is the commonest

mistake in this chapter.

6.1 Activation: aggregated and disaggregated form

(a) The link in words. A resource (a machine, a site, a plant) must be *activated* before being used. The binaries x_{ij} say "object i uses resource j", the binaries y_j say "resource j is activated". We want: if anyone uses j, then j is activated.

(b) The constraints. Two ways, with n objects and m resources:

$$\text{disaggregated:} \quad x_{ij} \le y_j, \qquad \forall i \in \{1,2,\dots,n\},\ \forall j \in \{1,2,\dots,m\} \qquad (n\,m \text{ constraints}), \tag{6.1}$$

$$\text{aggregated:} \quad \sum_{i=1}^{n} x_{ij} \le n\,y_j, \qquad \forall j \in \{1,2,\dots,m\} \qquad (m \text{ constraints}). \tag{6.2}$$

When the resource has a capacity k_j (at most k_j objects), the aggregated form is written with that coefficient, $\sum_i x_{ij} \le k_j y_j$, and then it imposes the capacity *as well*: two conditions in one constraint.

(c) The proof. On binary points the two forms are **equivalent**.

- *Disaggregated* $\Rightarrow$ *aggregated*: summing (6.1) over i gives $\sum_i x_{ij} \le n y_j$.
- *Aggregated* $\Rightarrow$ *disaggregated, on binary points*: if $y_j = 0$, (6.2) gives $\sum_i x_{ij} \le 0$ and hence $x_{ij} = 0 \le y_j$ for every i; if $y_j = 1$, then $x_{ij} \le 1 = y_j$ for every i because the x are binary.

The second direction uses the binarity of x and y: *outside* the binary points it does not hold, and that is exactly what makes the two forms different in the relaxation.

The converse is not imposed. "If j is activated then somebody uses it", that is $y_j = 1 \Rightarrow \sum_i x_{ij} \ge 1$, does **not** follow from the constraints: $y_j = 1$ with all $x_{ij} = 0$ is feasible in both forms. It follows from optimality when activating costs something: if the activation cost f_j is *strictly* positive, given an optimal solution with $y_j = 1$ and no $x_{ij} = 1$, setting $y_j = 0$ leaves all constraints satisfied and reduces the cost by $f_j > 0$ — a contradiction. Hence "in every optimum". If instead $f_j = 0$ is allowed, the correct conclusion is the weaker "there exists an optimum with $y_j = 0$".

(d) The strength of the relaxation. Instance: $n = 3$ customers, $m = 2$ sites, activation costs $f = (8, 6)$, service costs

$$c = \begin{pmatrix} 2 & 5 \\ 4 & 1 \\ 3 & 3 \end{pmatrix}.$$

Formulation	link constraints	$z(LP^+)$
aggregated (6.2)	$m = 2$	$38/3 \approx 12.67$
disaggregated (6.1)	$n\,m = 6$	15

The integer optimum is $z(MILP) = 15$: the disaggregated form already reaches it in the relaxation, the aggregated one stops at $38/3$. More rows, tighter relaxation: the typical trade-off of this technique. On large instances the choice is not obvious — $n\,m$ rows can be many — but on hard instances the disaggregated form almost always pays off.

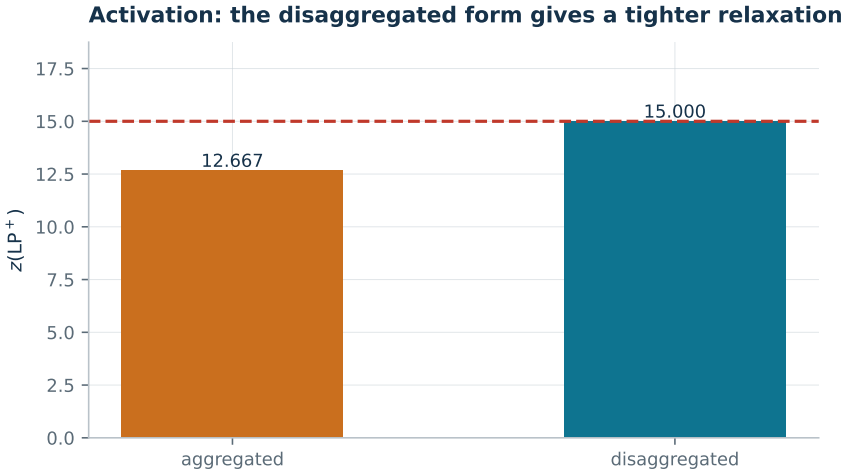


Figure 6.1: The relaxation of the two activation forms on the three-customer, two-site instance. The red line is the integer optimum.

(e) In gurobipy.

```
m.addConstrs((x[i, j] <= y[j] for i in range(n) for j in range(mm)),
    name="link")    # disaggregated
m.addConstrs((x.sum("*", j) <= n * y[j] for j in range(mm)), name="link
    ")            # aggregated
```

6.2 Fixed cost, capacity and continuous flow

(a) The link in words. As above, but the quantity used is *continuous*: $q_j \geq 0$ is how much is produced at plant j, and y_j says whether the plant is open. A closed plant produces nothing; an open one produces at most its capacity c_j.

(b) The constraints.

$$q_j \leq c_j\, y_j, \qquad \forall j \in \{1, 2, \ldots, m\} \qquad (m \text{ constraints}). \tag{6.3}$$

One constraint per plant, with the *right* coefficient: the capacity, not some large number picked at random.

(c) The proof. If $y_j = 0$ the constraint gives $q_j \leq 0$ and, together with $q_j \geq 0$, forces $q_j = 0$: the implication "closed ⇒ produces nothing" is imposed by the constraint, and with it its contrapositive "produces ⇒ open". If $y_j = 1$ the constraint gives $q_j \leq c_j$: the capacity. The direction "open ⇒ produces" is not imposed and follows from optimality only if $f_j > 0$, as in the previous section.

(d) The strength of the relaxation. Instance: two plants, fixed costs $f = (10, 14)$, unit costs $c = (3, 2)$, capacities $C = (6, 7)$, demand $D = 9$. The optimum is $z(MILP) = 44$ (both open, $q = (2, 7)$).

Coefficient in (6.3)	$z(LP^+)$
the capacity, c_j	$112/3 \approx 37.33$
a big-M $= 100$ (plus the constraint $q_j \le c_j$ separately)	$1059/50 = 21.18$

Same integer set, same optimum, relaxations far apart. The rule: **the coefficient of an activation binary is the smallest value the continuous variable can be capped at when the activation equals 1**, and it must be derived from the data.

(e) In gurobipy.

```
m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="link")
```

6.3 Minimum lot size and semicontinuous variables

(a) The link in words. "If you produce, produce at least ℓ": the quantity q_j is either zero or lies between a threshold ℓ and the capacity c_j. It is not an interval: it is the union of a point and an interval. A variable with this domain is called **semicontinuous**; when the quantity is integer on top of that — pieces, units, people — it is called **semi-integer**, and the two constraints are the same.

(b) The constraints. With the same binary y_j as in the previous section,

$$\begin{aligned} \ell\, y_j &\le q_j, \qquad \forall j \in \{1, 2, \dots, m\} \qquad (m \text{ constraints}), \\ q_j &\le c_j\, y_j, \quad \forall j \in \{1, 2, \dots, m\} \qquad (m \text{ constraints}). \end{aligned} \tag{6.4}$$

(c) The proof. Both directions are imposed by the constraints. If $y_j = 0$: $0 \le q_j \le 0$, that is $q_j = 0$. If $y_j = 1$: $\ell \le q_j \le c_j$. Hence

$$q_j \in \{0\} \cup [\ell,\ c_j],$$

exactly the domain wanted. One needs $\ell \le c_j$, otherwise $y_j = 1$ is infeasible and the variable is forced to zero: a data error the solver reports as infeasibility only if y_j is forced to 1 by other constraints.

(d) The strength of the relaxation. On the same instance as the previous section with $\ell = 5$: the optimum goes from 44 to $z(MILP) = 49$, with $q = (5, 5)$ instead of $(2, 7)$ — the threshold forces production of 5 at the second plant even though the first is more expensive. But $z(LP^+)$ stays at $112/3$, identical to the case without the threshold.

> In the relaxation y_j is free in $[0, 1]$, and the constraint $q_j \ge \ell y_j$ is satisfied by lowering y_j: $y_j \le q_j/\ell$ suffices. The threshold constraint *never bites* on the continuous problem. All of its effect is discharged onto integrality — which is why models with minimum lot sizes are typically harder than those without, at equal size.

(e) In gurobipy.

```
m.addConstrs((q[j] >= ell * y[j] for j in range(mm)), name="lot")
m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="capacity")
```

Gurobi also has the type `GRB.SEMICONT`, which declares the domain directly; in this course the formulation is written by hand, because that is what one must be able to prove.

6.4 Integer counts, multiple capacity, rounding up

(a) The link in words. "How many containers are needed?" The variable is not binary but *integer*: $w \in \mathbb{Z}_{\geq 0}$ counts indivisible objects (boxes, trucks, shifts, workers) and each carries a capacity c.

(b) The constraints.

$$\begin{aligned} \sum_i a_i\, x_i \;&\leq\; c\, w, && \text{(1 constraint)},\\ w \;&\in\; \mathbb{Z}_{\geq 0}. && \text{(1 integer variable)} \end{aligned} \tag{6.5}$$

(c) The proof. Setting $q = \sum_i a_i x_i$, the constraint imposes $w \geq q/c$ and integrality imposes $w \geq \lceil q/c \rceil$. Together with an objective that minimises w (or pays for w), in every optimum w equals exactly that ceiling: if it were larger, lowering it by 1 would stay feasible and reduce the cost. With a zero cost on w the conclusion weakens to "there exists an optimum".

> $\lceil t \rceil$ is not a linear function: it cannot be written inside a constraint. The pair "inequality $\leq cw$ + declaration that w is integer" realises it *implicitly*, and that is how it must be read when explaining the model.

(d) The strength of the relaxation. 17 unit items, capacity $c = 5$: the relaxation gives $w \geq 17/5 = 3.4$ and the integer optimum is $z(MILP) = 4$. The gap $4 - 17/5 = 3/5$ comes entirely from integrality: no linear cut on the x alone closes it, an inequality using w integer is needed.

(e) In gurobipy.

```
w = m.addVar(vtype=GRB.INTEGER, name="w")
m.addConstr(gp.quicksum(a[i] * x[i] for i in range(n)) <= c * w, name="
    capacity")
```

6.5 The maximum auxiliary variable

(a) The link in words. A variable z must equal the *maximum* of certain quantities: the duration of the longest activity, the highest connection cost, the load of the busiest machine.

(b) The constraints. If the quantities are $t_j x_j$ with x_j binary,

$$\begin{aligned} z \;&\geq\; t_j\, x_j, \quad \forall j \in \{1, 2, \ldots, n\} && (n \text{ constraints}),\\ z \;&\geq\; 0. \end{aligned} \tag{6.6}$$

(c) The proof, in three steps.

1. *Imposed by the constraint*: for every j with $x_j = 1$ we get $z \geq t_j$, hence $z \geq \max_{j:x_j=1} t_j$. This holds in every feasible solution.
2. *From optimality*: if z appears in the objective with a strictly positive coefficient in a minimisation (or a strictly negative one in a maximisation) and in no other constraint, then in every optimum z takes the smallest feasible value. Indeed, given an optimal solution with $z > \max_{j:x_j=1} t_j$, lowering z to that maximum leaves all constraints (6.6) satisfied and strictly improves the objective.
3. *Conclusion*: in every optimum, $z = \max_{j:x_j=1} t_j$ exactly (with the convention that the maximum over the empty set is 0, guaranteed by $z \geq 0$).

> If z appears in another constraint that wants it *large* — for instance a second constraint $z \geq w$, with w another variable — the exchange argument no longer works: lowering z may violate that constraint. In that case $z \geq \max$ still holds, but $z = \max$ does not. It is the typical mistake when the same auxiliary is reused for two purposes: if the maximum of two different families is needed, two auxiliaries are needed.

(d) The strength of the relaxation. $\min z$ with $z \geq t_j x_j$, $t = (4, 7, 3)$ and $\sum_j x_j \geq 2$: the integer optimum is $z(MILP) = 4$ (jobs 1 and 3 are chosen, and the maximum is 4). The relaxation is $168/61 \approx 2.75$: the fractional solution $x_j = z/t_j$ spreads the choice over all three jobs and lowers the maximum. The maximum link gives *weak* relaxations: one of the reasons makespan problems are hard.

(e) In gurobipy.

```
z = m.addVar(name="z")
m.addConstrs((z >= t[j] * x[j] for j in range(n)), name="maximum")
```

6.6 Min-max, max-min and the range

(a) The link in words. Three fairness objectives, often confused: *min-max* (minimise the largest load), *max-min* (maximise the smallest load) and *range* (minimise the gap between the largest and the smallest).

(b) The constraints. With ℓ_k the load of resource k ($k \in \{1, 2, \ldots, m\}$):

$$\text{min-max:} \quad \min z \quad \text{with} \quad z \geq \ell_k, \ \forall k \in \{1, 2, \ldots, m\} \qquad (m \text{ constraints}), \tag{6.7}$$

$$\text{max-min:} \quad \max u \quad \text{with} \quad u \leq \ell_k, \ \forall k \in \{1, 2, \ldots, m\} \qquad (m \text{ constraints}), \tag{6.8}$$

$$\text{range:} \quad \min (z - u) \quad \text{with both} \qquad (2m \text{ constraints}). \tag{6.9}$$

(c) The proof. Each is the maximum (or minimum) auxiliary variable of the previous section, with the exchange argument in the right direction: in a $\min z$ the variable z falls to the maximum of the loads; in a $\max u$ the variable u rises to the minimum of the loads. In the range both pressures are present at once and the two conclusions hold together.

On the five-weight instance $p = (3, 5, 2, 4, 7)$ to be split between two workers (total 21), the three versions choose the *same* split $(11, 10)$ — the best possible, because the total is odd — but their optimal values are 11, 10 and 1. They are three different numbers describing the same solution. Comparing "$z = 11$" of a min-max with "$z = 1$" of a range means nothing: before comparing two models one must make sure they optimise the same thing. And the optimal solutions need not coincide either: with more than two resources, min-max and max-min in general choose different splits.

(d) The strength of the relaxation. The min-max on that instance gives $z(LP^+) = 21/2 = 10.5$ against $z(MILP) = 11$: the relaxation splits the weights exactly in half, which integrality does not allow.

(e) In gurobipy.

```
z = m.addVar(name="z")
m.addConstrs((z >= load[k] for k in range(len(load))), name="max")
m.setObjective(z, GRB.MINIMIZE)
```

6.7 The absolute value

(a) The link in words. One wants $|u - v|$: a gap, an error, an imbalance. The case in the objective and the case in a constraint behave radically differently.

(b) The constraints.

- **In the objective, minimising**: a variable $d \geq 0$ and two constraints,

$$\begin{aligned} d &\geq u - v, && \text{(1 constraint)}, \\ d &\geq v - u, && \text{(1 constraint)}, \end{aligned} \tag{6.10}$$

  with d in the objective to be minimised. No binary.

- **As a $\leq$ constraint**: $|u - v| \leq k$ is simply $u - v \leq k$ and $v - u \leq k$ (2 constraints, no binary).

- **As a $\geq$ constraint**: $|u - v| \geq k$ **cannot** be written without binaries. It is the disjunction "$u - v \geq k$ *or* $v - u \geq k$", and needs a binary b and a big-M:

$$\begin{aligned} u - v &\geq k - M(1 - b), && \text{(1 constraint)}, \\ v - u &\geq k - M\,b. && \text{(1 constraint, 1 binary)} \end{aligned} \tag{6.11}$$

(c) The proof. In the first case the two constraints (6.10) impose $d \geq |u - v|$ (one of the two right-hand sides *is* $|u - v|$); the objective, which minimises d and in which d appears nowhere else, drives it to equality by the exchange argument of Section 6.5. In the third case, $b = 1$ switches off the second constraint (provided $M \geq k + \max(v - u)$) and leaves the first, and vice versa: it is a disjunction, not a conjunction, and without the binary both would be imposed — that is, $0 \geq 2k$, infeasible for $k > 0$.

$|u - v| \le k$ is the intersection of two half-planes: a convex set, written with two linear constraints. $|u - v| \ge k$ is the complement of a strip: it is *not* convex, and no system of linear constraints without integer variables can describe it. The binary is not a trick: it is necessary.

(d) The strength of the relaxation. On the five-weight instance, the problem min $|\ell_1 - \ell_2|$ has optimum $z(MILP) = 1$ and relaxation $z(LP^+) = 0$: the continuous problem splits 21 into two equal halves and zeroes the gap. The relaxation of an absolute-value objective is typically 0, that is, useless.

(e) In gurobipy.

```
d = m.addVar(name="d")
m.addConstr(d >= u - v, name="abs_plus")
m.addConstr(d >= v - u, name="abs_minus")
```

6.8 Big-M: conditional constraints and disjunctions

(a) The link in words. "If $y = 1$ then the constraint $a'x \le b$ holds; if $y = 0$ the constraint is not there." A constraint that switches on and off.

(b) The constraint.

$$a'x \;\le\; b + M\,(1 - y) \qquad \text{(1 constraint)}. \tag{6.12}$$

(c) The proof, and how M is chosen. With $y = 1$ the constraint is $a'x \le b$: switched on. With $y = 0$ it becomes $a'x \le b + M$: switched off *provided* M is large enough to cut nothing off, that is

$$M \;\ge\; \max\{a'x : x \text{ feasible for the other constraints}\} - b. \tag{6.13}$$

Three things must be told apart:

- a **valid** M: it satisfies (6.13), so the model has the right integer set;
- an **improvable** M: valid but larger than necessary; the integer set is the same, the relaxation is weaker;
- the **smallest proved** M: the smallest value for which validity has been *proved*, typically by computing the maximum in (6.13) from the variable bounds alone. It is not necessarily the absolute minimum, and it is honest to say so.

With $a = (3, 4, 5)$, $b = 6$ and x binary, the maximum of $a'x$ is 12, so $M = 12 - 6 = 6$ is valid. On max $x_1 + x_2 + x_3 + y$ the optimum is $z(MILP) = 3$ (with $y = 0$ and all the x at 1). With $M = 5$ the constraint with $y = 0$ remains $3x_1 + 4x_2 + 5x_3 \le 11$, which excludes $x = (1, 1, 1)$: the optimum drops to 2. The model no longer answers the question asked.

(d) The strength of the relaxation. On the same instance:

M	$z(LP^+)$	
6 (the smallest proved)	3	coincides with $z(MILP)$
20	$37/10 = 3.7$	
1000	$1997/500 \approx 3.994$	almost the largest possible, 4

The degradation is monotone and fast. The operational rule: **compute M from the data, always, and write it in the text right after the model**.

(e) In gurobipy.

```
M = sum(max(a[j], 0) for j in range(n)) - b        # computed from the data, not guessed
m.addConstr(gp.quicksum(a[j] * x[j] for j in range(n)) <= b + M * (1 - y), name="cond")
```

6.9 Precedences and sequencing

(a) The link in words. On a machine that runs one job at a time, for every pair of jobs one precedes the other. It is a disjunction per pair, not a precedence fixed by the data.

(b) The constraints. With $\kappa_j \geq 0$ the completion time of job j, t_j its duration and $s_{ij} \in \{0,1\}$ equal to 1 if j precedes i:

$$s_{ij} + s_{ji} = 1, \qquad \forall i,j \in \{1,2,\dots,n\},\ i<j \qquad \left(\binom{n}{2} \text{ constraints}\right), \tag{6.14}$$

$$\kappa_i \geq \kappa_j + t_i - M\,(1 - s_{ij}), \qquad \forall i,j \in \{1,2,\dots,n\},\ i \neq j \qquad (n(n-1) \text{ constraints}), \tag{6.15}$$

$$\kappa_j \geq t_j, \qquad \forall j \in \{1,2,\dots,n\} \qquad (n \text{ constraints}). \tag{6.16}$$

(c) The proof, and the smallest M. Constraint (6.14) imposes that exactly one of the two orders is chosen. If $s_{ij} = 1$, (6.15) becomes $\kappa_i \geq \kappa_j + t_i$: job i finishes at least t_i after j finishes, so they do not overlap. If $s_{ij} = 0$ it becomes $\kappa_i \geq \kappa_j + t_i - M$, which must always hold. Since $\kappa_i \geq t_i$ and $\kappa_j \leq \sum_h t_h$ in every sensible solution, it suffices to take

$$M \;=\; \sum_{h=1}^{n} t_h,$$

because then $\kappa_j + t_i - M \leq \sum_h t_h + t_i - \sum_h t_h = t_i \leq \kappa_i$. The time horizon $\sum_h t_h$ must be *declared*: without an upper bound on the κ_j, no finite M is valid.

(d) The strength of the relaxation. Three jobs of duration $(3,2,4)$ on one machine, makespan objective: $z(MILP) = 9 = \sum_h t_h$ (obviously: a single machine), with completion times $(3,5,9)$. The relaxation is 4: with $s_{ij} = 1/2$ all the precedence constraints are half switched off and the jobs may overlap. It is the weakest relaxation in the whole chapter, and it explains why big-M sequencing models scale badly.

(e) In gurobipy.

```
M = sum(t)                                          # the horizon, declared
for i in range(n):
    for j in range(i):
        m.addConstr(s[i, j] + s[j, i] == 1, name=f"disj{i}{j}")
        m.addConstr(kappa[i] >= kappa[j] + t[i] - M * (1 - s[i, j]),
            name=f"prec{i}{j}")
        m.addConstr(kappa[j] >= kappa[i] + t[j] - M * (1 - s[j, i]),
            name=f"prec{j}{i}")
```

6.10 "If and only if"

(a) The link in words. A binary y must equal 1 *exactly when* a condition holds — for instance "all the jobs of the class are done". One implication is not enough: two are needed.

(b) The constraints. With $x_1, \dots, x_p$ binary and the condition "all equal 1":

$$y \le x_j, \qquad \forall j \in \{1, 2, \dots, p\} \qquad (p \text{ constraints}), \tag{6.17}$$

$$y \ge \sum_{j=1}^{p} x_j - (p-1) \qquad (1 \text{ constraint}). \tag{6.18}$$

(c) The proof, in both directions.

- (6.17) gives $y = 1 \Rightarrow x_j = 1$ for every j; equivalently, if even one of the x_j is zero then $y = 0$.
- (6.18) gives the converse: if all $x_j = 1$, the right-hand side equals $p-(p-1) = 1$, so $y \ge 1$ and hence $y = 1$. If at least one is zero, the right-hand side is ≤ 0 and the constraint says nothing.

Together, the two impose $y = 1 \iff \sum_j x_j = p$ in *every* feasible solution.

When one direction is enough. If y appears in the objective of a maximisation with a bonus $v > 0$ and in no other constraint, direction (6.18) may be omitted: in every optimum, if all $x_j = 1$ and $y = 0$, raising y to 1 stays feasible for (6.17) and increases the objective by $v > 0$. This is an optimality argument, and it holds only at the optimum.

On the three-job instance, revenues $(2, 2, 2)$, capacity 3:

bonus v	constraints	$z(MILP)$	y, x at the optimum	does y say "class complete"?
9	only (6.17)	15	$y = 1$, $x = (1, 1, 1)$	yes
9	both	15	$y = 1$, $x = (1, 1, 1)$	yes
0	only (6.17)	6	$y = 0$, $x = (1, 1, 1)$	**no**
0	both	6	$y = 1$, $x = (1, 1, 1)$	yes

With $v = 0$ the optimal value is the same in the two models, but in the third case y is no longer a faithful indicator. If y only serves to collect a bonus, the missing direction is

useless; if y appears in *other* constraints — and this happens as soon as a condition on the number of complete classes is added — it must be written.

(d) The strength of the relaxation. $z(LP^+) = 15 = z(MILP)$ on this instance: with both directions imposed the relaxation is exact. Constraint (6.18) is weak in the relaxation (with $x_j = 1/2$ its right-hand side is negative), but here that does not matter.

(e) In gurobipy.

```
m.addConstrs((y <= x[j] for j in range(p)), name="iff_up")
m.addConstr(y >= gp.quicksum(x[j] for j in range(p)) - (p - 1), name="
    iff_down")
```

6.11 Counting the different types

(a) The link in words. "At least two different types must be produced", "at most three foods in the diet", "no more than four configurations". One counts *how many* elements are active, not how much of them is produced.

(b) The constraints. With $q_j \geq 0$ the quantity and y_j the indicator:

$$q_j \leq c_j\, y_j, \qquad \forall j \in \{1, 2, \ldots, n\} \qquad (n \text{ constraints}), \tag{6.19}$$

$$q_j \geq \ell\, y_j, \qquad \forall j \in \{1, 2, \ldots, n\} \qquad (n \text{ constraints}), \tag{6.20}$$

$$\sum_{j=1}^{n} y_j \geq p \qquad (1 \text{ constraint}). \tag{6.21}$$

(c) The proof, and why both thresholds are needed. (6.19) gives "$y_j = 0 \Rightarrow q_j = 0$"; (6.20) gives the missing direction "$y_j = 1 \Rightarrow q_j \geq \ell$". Without (6.20), a solution with $y_j = 1$ and $q_j = 0$ is feasible: the counting constraint (6.21) is satisfied by switching on empty indicators, and the condition "at least p different types" says nothing any more. **Counting types works only if every switched-on type actually produces something**, and the threshold ℓ is what guarantees it.

(d) The strength of the relaxation. Three types, unit revenues $(4, 3, 5)$, resource 12, capacity 10 each, threshold $\ell = 3$, at least two types: $z(MILP) = 57$, with $q = (3, 0, 9)$. The relaxation is also 57: here the counting introduces no gap at all.

(e) In gurobipy.

```
m.addConstrs((q[j] <= c[j] * y[j] for j in range(n)), name="activate")
m.addConstrs((q[j] >= ell * y[j] for j in range(n)), name="lot")
m.addConstr(y.sum() >= p, name="at_least_p_types")
```

6.12 Alldiff and binary expansion

(a) The link in words. Two different but related needs: giving n objects n *all distinct* values (*alldiff*), and representing a bounded integer variable with binary variables (*binary expansion*).

(b) The constraints. *Alldiff*, with $p_{iv} = 1$ if object i gets value v:

$$\sum_{v=1}^{n} p_{iv} = 1, \qquad \forall i \in \{1, 2, \dots, n\} \qquad (n \text{ constraints}), \tag{6.22}$$

$$\sum_{i=1}^{n} p_{iv} = 1, \qquad \forall v \in \{1, 2, \dots, n\} \qquad (n \text{ constraints}). \tag{6.23}$$

Binary expansion of $v \in \{0, 1, \dots, 2^p - 1\}$:

$$\begin{aligned} v &= \sum_{k=0}^{p-1} 2^k \, b_k, && (1 \text{ constraint}), \\ b_k &\in \{0, 1\}, \qquad \forall k \in \{0, 1, \dots, p-1\} && (p \text{ binaries}). \end{aligned} \tag{6.24}$$

(c) The proof. Alldiff is a double *set partitioning*: the first group gives "every object one value", the second "every value to one object". Together they impose a bijection, that is, all distinct values. Binary expansion is the base-2 representation, unique for every integer in that range: the correspondence between v and $(b_0, \dots, b_{p-1})$ is one-to-one.

Alldiff has an exact relaxation, expansion does not help

The matrix of the double partitioning is the one of the assignment problem: it is totally unimodular, so every vertex of the relaxation is integer and $z(LP^+) = z(MILP)$. On the 3×3 instance of the script both are 7: integrality is free. Binary expansion, by contrast, adds no strength: $\sum_k 2^k b_k$ with $b_k \in [0, 1]$ covers all of $[0, 2^p - 1]$ continuously, exactly like $v \geq 0$, $v \leq 2^p - 1$. It serves to *reformulate*, not to tighten — for instance when another part of the model needs binary indicators rather than an integer variable.

(d) The example. $v \in \{0, \dots, 7\}$ with $v \geq 5$, $\min v$: the model gives $v = 5 = 1 + 4$, that is $(b_0, b_1, b_2) = (1, 0, 1)$.

(e) In gurobipy.

```
m.addConstrs((p.sum(i, "*") == 1 for i in range(n)), name="one_value")
m.addConstrs((p.sum("*", v) == 1 for v in range(n)), name="alldiff")
m.addConstr(v == gp.quicksum(2 ** k * b[k] for k in range(p)), name="
    expansion")
```

6.13 Soft constraints, deviations and penalties

(a) The link in words. A constraint one would *prefer* to satisfy, but which may be violated at a price: the demand of a period, a timetable preference, a service target. Turning a hard constraint into a soft one is often the difference between an infeasible model and a useful one.

(b) The constraint. In place of $a'x = \beta$ one writes

$$\begin{aligned} a'x + s^- - s^+ &= \beta, && (1 \text{ constraint}), \\ s^- &\geq 0, && (1 \text{ continuous variable}), \\ s^+ &\geq 0, && (1 \text{ continuous variable}), \end{aligned} \tag{6.25}$$

and adds $\pi^- s^- + \pi^+ s^+$ to the objective, with penalties $\pi^-, \pi^+ > 0$. The variable s^- measures how far *below* the target one is, s^+ how far above.

(c) The proof. The constraint alone is always satisfiable: any $a'x$ can be compensated with one of the two deviations. The interesting property is that **in every optimum at least one of the two deviations is zero**: if $s^- = \sigma > 0$ and $s^+ = \tau > 0$, subtracting $\min(\sigma, \tau)$ from both leaves (6.25) satisfied (the difference $s^- - s^+$ does not change) and reduces the objective by $(\pi^- + \pi^+)\min(\sigma, \tau) > 0$. So s^- and s^+ really are the positive and negative parts of the deviation, and $s^- + s^+ = |a'x - \beta|$ in every optimum: the same thing as Section 6.7, written with an equality instead of two inequalities.

> The exchange argument only needs $\pi^- + \pi^+ > 0$. With $\pi^- = 0$ and $\pi^+ > 0$ every optimum still has at least one deviation equal to zero, but falling short of β costs nothing: the soft constraint no longer pushes $a'x$ up towards the target. Only if $\pi^- + \pi^+ \leq 0$ may an optimal solution have both deviations positive (with a negative sum the model is even unbounded), and the reading "s^- is the shortfall" is lost. The same rule as everywhere in this chapter: the strength of the conclusion depends on the sign of the coefficients in the objective.

(d) The example. Demand 6 in each of three periods, total availability $15 < 18$, penalties $\pi^+ = 3$ and $\pi^- = 2$. The hard model would be infeasible; the soft one gives $z(MILP) = 6$, with $q = (3, 6, 6)$ and a shortfall of 3 concentrated in the first period. Nothing in the data says it should be concentrated: any split of the total shortfall 3 has the same cost, and the solver returns one of them.

(e) In gurobipy.

```
sm = m.addVars(T, name="s_minus");  sp = m.addVars(T, name="s_plus")
m.addConstrs((q[t] + sm[t] - sp[t] == demand[t] for t in range(T)),
    name="target")
m.setObjective(cost + gp.quicksum(pen_down * sm[t] + pen_up * sp[t] for
     t in range(T)),
                GRB.MINIMIZE)
```

6.14 Piecewise linear functions

(a) The link in words. The cost is not proportional to the quantity: it changes in brackets — discounts above a threshold, surcharges beyond a capacity, banded tariffs. The function $g(q)$ is continuous and piecewise linear, with breakpoints $q_0 < q_1 < \cdots < q_K$ and values $g_0, \ldots, g_K$.

(b) The constraints. With $\lambda_k \geq 0$ the weights of the convex combination and $w_t \in \{0,1\}$ the indicator of piece t (between q_{t-1} and q_t):

$$\sum_{k=0}^{K} \lambda_k = 1, \tag{6.26}$$

$$q = \sum_{k=0}^{K} q_k \lambda_k, \qquad \text{(1 constraint)}, \tag{6.27}$$

$$g(q) = \sum_{k=0}^{K} g_k \lambda_k, \qquad \text{(1 constraint)}, \tag{6.28}$$

$$\sum_{t=1}^{K} w_t = 1, \qquad \text{(1 constraint)}, \tag{6.29}$$

$$\lambda_k \leq \sum_{t\,:\,k \in \{t-1,t\}} w_t, \qquad \forall k \in \{0,1,\dots,K\} \qquad (K+1 \text{ constraints}). \tag{6.30}$$

(c) The proof. Constraint (6.30) is **adjacency**: λ_k can be positive only if the chosen piece has q_k as an endpoint. With (6.29) the piece is unique, so at most two consecutive λ are positive, and then $(q, g(q))$ is a point of the segment between (q_{t-1}, g_{t-1}) and (q_t, g_t): that is, a point *of the graph*. Without adjacency one could mix non-consecutive nodes, and (q, g) would land in the **lower convex envelope** of the graph — below the function.

When g is convex and one *minimises*, the lower convex envelope coincides with the function and the optimal λ turn out adjacent automatically: the model without binaries is already correct. When g is not convex — the case of quantity discounts, where marginal cost *falls* — the difference shows immediately. With breakpoints $(0, 4, 10, 16)$, values $(0, 12, 30, 36)$ and demand $q \geq 13$:

Formulation	optimal value
without adjacency (free convex combination)	$117/4 = 29.25$
with adjacency (6.30)	33

The first mixes nodes 0 and 3 (the only nonzero λ) and returns a cost the function *attains nowhere*: it is the value of the envelope, not of the graph. The second mixes nodes 2 and 3, adjacent, and gives the exact value $g(13) = 30 + 6 \cdot \frac{3}{6} = 33$.

(d) The strength of the relaxation. Two things must not be confused: adjacency changes the *integer set* (the two models have different optima, 29.25 and 33), but not the strength of the relaxation — with w_t fractional, constraint (6.30) does not bite and the two models share the same $z(LP^+) = 117/4$.

(e) In gurobipy.

```
m.addConstr(lam.sum() == 1, name="convex")
m.addConstr(q == gp.quicksum(nodes[k] * lam[k] for k in range(K + 1)),
    name="abscissa")
m.addConstr(w.sum() == 1, name="one_piece")
```

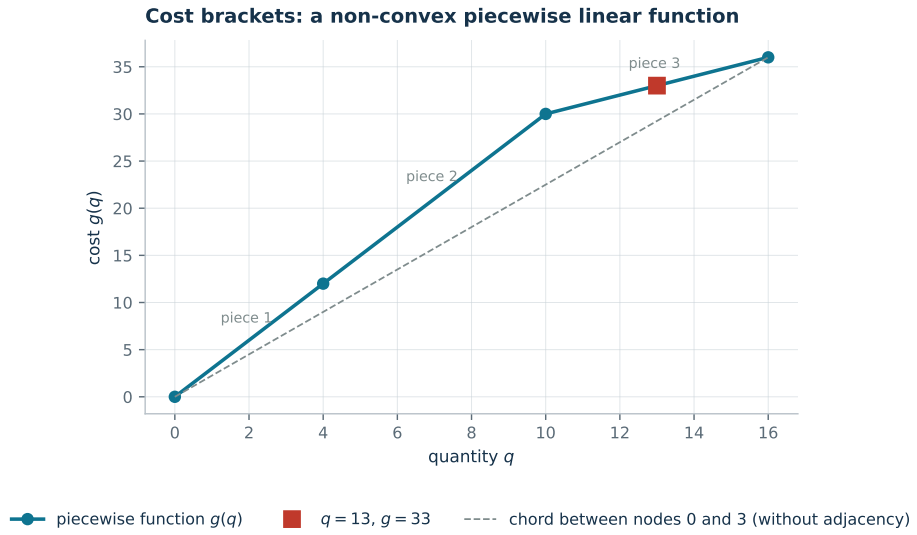


Figure 6.2: The piecewise function with a quantity discount. The dashed chord between the extreme nodes is what the formulation without adjacency can reach: it lies *below* the graph.

```
for k in range(K + 1):                          # adjacency: lambda_k only
    on the pieces touching k
    m.addConstr(lam[k] <= gp.quicksum(w[t] for t in (k - 1, k) if 0 <=
        t < K),
                name=f"adjacency{k}")
```

Gurobi also offers `addGenConstrPWL` and SOS2 types, which do the same job internally; here the manual formulation remains the main material, because it is the one whose correctness must be provable.

6.15 The map of techniques

The following table is meant for consultation: read from the first column ("what I want to say") to the second ("how it is written"), while the third recalls what must be declared every time the technique is used.

Technique	Formulation	To be declared
6.1 activation	$x_{ij} \le y_j$ (disagg.) or $\sum_i x_{ij} \le k_j y_j$ (agg.)	which form, and why; sign of f_j for the optimality direction
6.2 fixed cost	$q_j \le c_j y_j$	that c_j is the capacity, not a big-M
6.3 minimum lot	$\ell y_j \le q_j \le c_j y_j$	that $\ell \le c_j$; that the relaxation is unaffected
6.4 integer count	$\sum_i a_i x_i \le K w$, w integer	that integrality realises the ceiling
6.5 maximum auxiliary	$z \ge t_j x_j$, $z \ge 0$	that z appears nowhere else; its sign in the objective
6.6 min-max / max-min	$T \ge L_k$ and $\min T$; $U \le L_k$ and $\max U$	which of the three objectives, and that they are not comparable
6.7 absolute value	$d \ge \pm(u - v)$ in the objective; disjunction if $\ge k$	whether it is objective or constraint, and in which direction
6.8 big-M	$a'x \le b + M(1 - y)$	the value of M computed from the data
6.9 precedences	$s_{ij}+s_{ji} = 1$, $\kappa_i \ge \kappa_j + t_i - M(1-s_{ij})$	the horizon and $M = \sum_h t_h$
6.10 if and only if	$y \le x_j$ and $y \ge \sum_j x_j - (p-1)$	whether the second direction is needed or follows from optimality
6.11 counting types	$\ell y_j \le q_j \le c_j y_j$, $\sum_j y_j \ge p$	that without the threshold ℓ the count is empty
6.12 alldiff / expansion	double partitioning; $v = \sum_k 2^k b_k$	that alldiff has an exact relaxation
6.13 soft constraints	$a'x + s^- - s^+ = \beta$ with penalties	the two signs of the penalties
6.14 piecewise function	convex combination + adjacency	whether g is convex; otherwise adjacency is mandatory

The three questions to ask about a new link

1. **Which direction is imposed by the constraints and which is not?** Answered by looking at the constraints alone, one case per value of the binary.
2. **Does the missing direction follow from optimality?** Answered by the exchange argument, and the answer depends on the *sign* of the coefficient in the objective: strictly nonzero gives "in every optimum", zero gives at most "there exists an optimum".
3. **How strong is the relaxation?** Answered by comparing $z(LP^+)$ with $z(MILP)$ on a small instance and — when there are two formulations — by comparing them with each other. The two formulations must first be proved equivalent on integer points: without that step one is not comparing two models of the same problem.

Index

absolute value, 8
activation, 20, 28, 59, 61

big-M, 64, 66
bin packing, 24, 37
binary expansion, 68

certified gap, 9, 15, 20, 39
constructive heuristic, 15, 17, 36, 40, 42, 43

dual bound, 8, 10, 13, 18, 39

exchange argument, 58, 63, 64, 70, 73

integrality gap, 44

knapsack, 19, 21, 23, 40

LP relaxation, 6, 7
LPT, 38

minimum lot, 61

nearest neighbour, 40

primal bound, 9, 36, 43

scheduling on identical machines, 38
semi-continuous variable, 61
semi-integer variable, 61
set covering, 9, 14, 57
set packing, 9, 57
set partitioning, 9, 57, 69
strong duality, 8, 13, 15

tolerances, 9
travelling salesman problem, 24, 40

valid inequalities, 22